

OpenFrame マイグレーションガイド

Base 7.1, Batch for MVS 7.1, TACF 7.1, HiDB 7.2, OSC 7.1, OSI 7.2



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame マイグレーションガイド

発行日: 2023年12月29日

ソフトウェアバージョン: Base 7.1、Batch for MVS 7.1、TACF 7.1、HiDB 7.2、OSC 7.1、OSI 7.2

ガイドバージョン: v3.1.3

Website

<https://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 紹介	1
1.1. リホストと移行	1
1.2. 移行のタイプ	1
1.2.1. リソースの移行	2
1.2.2. データセットの移行	3
1.2.3. アプリケーションの移行	3
1.2.4. データベースの移行	4
1.3. 移行プロセス	4
第2章 リソースの移行	5
2.1. EBCDICからASCIIへの変換	5
2.2. CICSマップの移行	8
2.3. CICSリソースの移行	10
2.4. JCLの移行	11
2.4.1. DB2	11
第3章 データセットの移行	15
3.1. 概要	15
3.2. 非VSAMデータセットの移行	17
3.2.1. 移行対象の確認およびソース・ファイルのダウンロード	17
3.2.2. データセット・レイアウトの分析	17
3.2.3. データセット変換スキーマの作成	18
3.2.4. ターゲット・データセットの移行	20
3.2.5. PDSの移行	20
3.2.6. GDGベースの定義	21
3.2.7. 可変長データセットの移行	21
3.2.8. データセット移行の検証	22
3.3. VSAMデータセットの移行	23
3.3.1. VSAMデータセットの定義	23
3.3.2. VSAMデータセットのインポート	24
3.3.3. データセット・インポート・プログラムの作成	24
3.4. 静的変換と動変換	24
3.4.1. 静的変換と動変換の比較	25
3.4.2. 動変換のみ使用できる場合	25
3.4.3. 動変換の使用を推奨する場合	25
3.4.4. 動変換の使用方法と処理順序	26
3.5. データセットの移行	27
3.5.1. 固定長データセット	27
3.5.2. 可変長データセット	28
3.5.3. REDEFINES文が使用されたデータセット	29

3.5.4. 複数のレイアウトを使用するデータセット	31
第4章 アプリケーションの移行	33
4.1. CICSアプリケーションの移行	33
4.1.1. 前処理	33
4.1.2. コンパイル	35
4.1.3. デプロイ	35
4.2. バッチ・アプリケーションの移行	36
4.2.1. COBOLバッチ・アプリケーション	37
4.2.2. PL/Iバッチ・アプリケーション	39
付録 A. EBCDICからASCIIへの変換の注意点	41
A.1. 16進数値の処理	41
A.2. 文字ソート順の処理	43
A.3. 2バイト・スペースの処理	46
付録 B. DB2からOracleへの移行の注意点	47
付録 C. スキーマ・ファイルの構造	53
C.1. スキーマ・ファイルの構造(v7.1)	53
C.1.1. ODO(Occurs Depending On)項目	55
C.1.2. REDEFINES文	56
C.1.3. 条件値(VALUE)のタイプ	57
付録 D. TSAM RDB	59
D.1. VSAMデータセットの定義プロセス	59
D.2. TSAM RDBの特徴	59
D.3. 使用例	63
索引	65

図目次

[図 1.1]	OpenFrameの移行プロセス	2
[図 1.2]	リソースの移行	2
[図 1.3]	データセットの移行	3
[図 1.4]	アプリケーションの移行	4
[図 3.1]	データセットの移行プロセス	16
[図 3.2]	可変長データセットのソース・ファイル	22
[図 3.3]	devviewツールを使用したデータセットの検索	23
[図 3.4]	固定長データセットのソース・ファイル	27
[図 3.5]	固定長データセットの移行結果	28
[図 3.6]	可変長データセットのソース・ファイル	28
[図 D.1]	VSAM データセットの定義プロセス	59

このガイドについて

対象読者

本書は、Tmax OpenFrame[®](以下、OpenFrame)の移行ツールを使用して、メインフレーム環境のソースをOpenFrame環境に移行するエンジニアを対象としています。

前提知識

本書を理解するには、以下についての知識が必要です。

- メインフレーム
- OpenFrame/BatchおよびOpenFrame/OSC
- DB2およびOracle
- ユーザー・システムで使用されているサードパーティ・ユーティリティ

本書の構成

本書は、計4章と付録で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: 紹介

OpenFrameを利用した移行と移行のタイプについて説明します。

- 第2章: リソースの移行

CICSマップ、CICSリソース、JCLを移行する方法について説明します。

- 第3章: データセットの移行

データセットを移行する方法について説明します。

- 第4章: アプリケーションの移行

CICSアプリケーションとバッチ・アプリケーションを移行する方法について説明します。

- 付録A: EBCDICからASCIIへの変換の注意点

EBCDICコードをASCIIに変換するときの注意事項について説明します。

- 付録B: DB2からOracleへの移行の注意点

DB2をOracleに移行するときの注意事項について説明します。

- 付録C: スキーマ・ファイルの構造

スキーマ・ファイルの構造と形式について説明します。

- 付録D: TSAM RDB

データをコピーブック・フィールドごとに変換して管理できるようにVSAMデータセットを定義する機能について説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

関連文書

ガイド	説明
OpenFrame Baseガイド	OpenFrame/Baseシステムの構成要素と全般的な機能について説明しています。
OpenFrame Batchガイド	OpenFrame/Batchシステムの構成要素と全般的な機能について説明しています。
OpenFrame データセットガイド	OpenFrameデータセットを紹介し、データセットの種類やカタログ方法などについて説明しています。
OpenFrame TJESガイド	TJESを使用してOpenFrameシステムのバッチ・ジョブを管理する方法について説明しています。
OpenFrame OSCリソース定義ガイド	OpenFrame/OSCシステムを使用するために必要なリソース定義について説明しています。
OpenFrame OSCマッピングサポートガイド	TN3270端末との通信インターフェースであるマッピング・サポートについて説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameエンジンと共に提供される多様なユーティリティ・プログラムについて説明しています。
OpenFrame エラーメッセージ リファレンスガイド	OpenFrame製品の使用中に発生し得るエラーのタイプおよびその解決方法について説明しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用に使用する多様なツール・プログラムについて説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

第1章 紹介

本章では、OpenFrameのの移行について紹介し、移行のタイプについて説明します。

1.1. リホストと移行

メインフレームの業務システムをオープン環境に移行する方法には、以下の2つの方法があります。

- ビッグバン(Big-Bang)

既存の業務システムを分析した後、オープン環境に合わせて再設計および再開発する方法です。

- リホスト(Rehosting)

リホスト・ソリューションを使用してメインフレームの業務システムをオープン環境でそのまま実行できる方法です。

ビックバンは、膨大な工数を要する方法ですが、リホストは、ソリューションを使用して業務ロジックを移行できるため、少ない工数でオープン環境への移行を可能にします。

OpenFrameは、最も効率的なリホスト・ソリューションであり、メインフレームのリソースをOpenFrameで使用できるように移行することを**OpenFrame移行**といいます。

1.2. 移行のタイプ

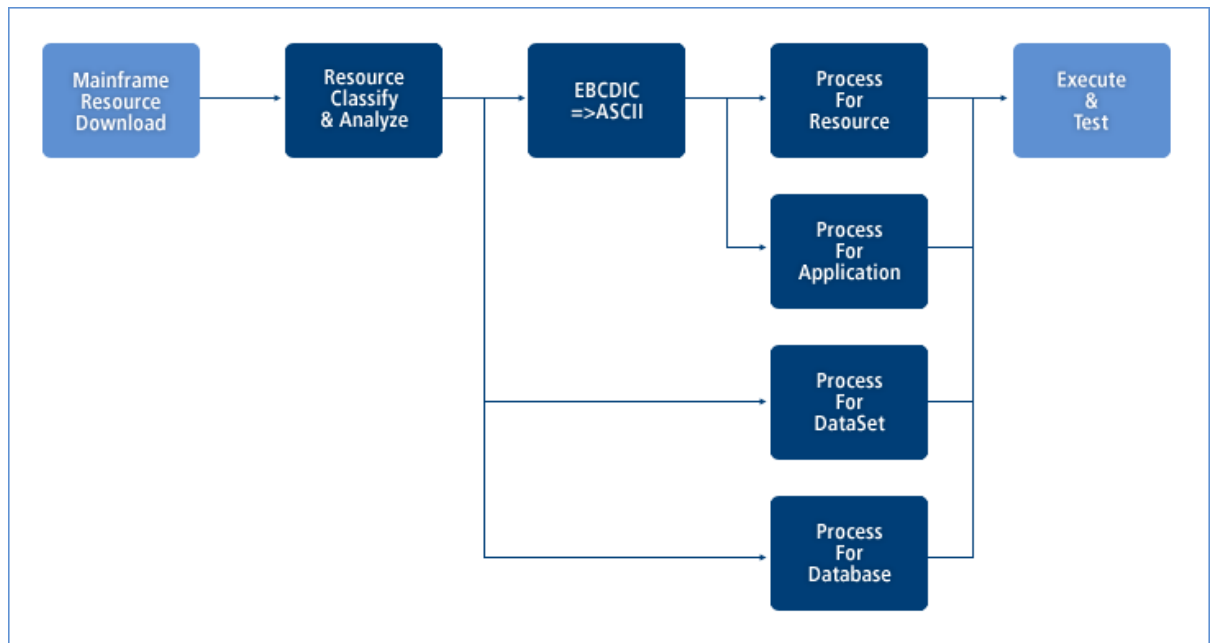
OpenFrameでの移行には、次の4つのタイプがあります。

- リソースの移行
- データセットの移行
- アプリケーションの移行
- データベースの移行

OpenFrameの移行を4つのタイプに分類していますが、実際の移行では、関連するリソース、データセット、アプリケーション、データベース単位で一緒に移行されます。

以下の図は、一般的なOpenFrameの移行プロセスを示します。

[図 1.1] OpenFrameの移行プロセス

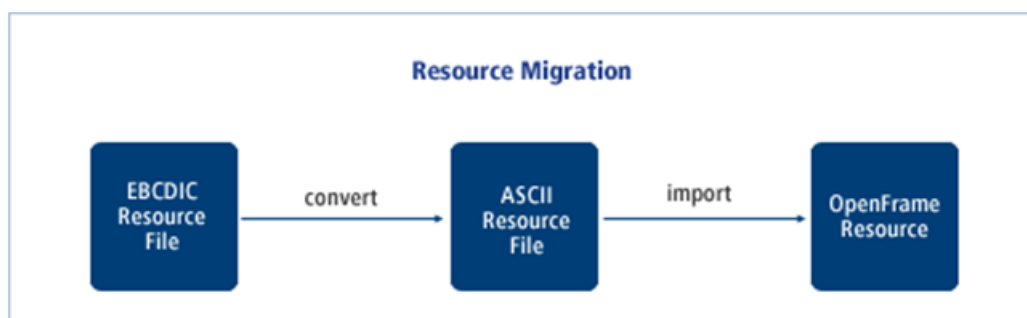


1.2.1. リソースの移行

リソースの移行は、メインフレームの業務システムで使用されているJCLやCSDなどのユーザー・リソースを、OpenFrameで使用できるように移行する作業です。リソースを移行するには、メインフレームからダウンロードしたEBCDICデータが必要です。

以下の図は、一般的なリソースの移行プロセスを示します。

[図 1.2] リソースの移行



1. EBCDICリソースをASCIIリソースに変換します。
2. 変換したASCIIリソースをOpenFrameにインポートします。

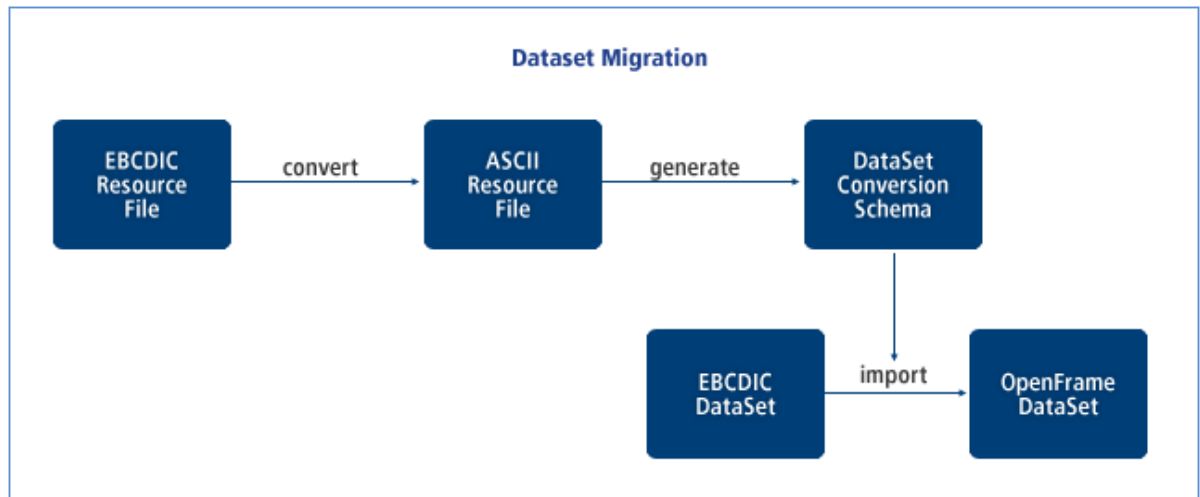
リソースは、OpenFrameシステムで使用するすべてのリソースを意味しますが、本書では、データセット、アプリケーション、その他のリソースに分類し、「第2章：リソースの移行」、「第3章：データセットの移行」、「第4章：アプリケーションの移行」で説明しています。

1.2.2. データセットの移行

データセットの移行は、メインフレームで使用していたデータセットをOpenFrameで使えるように移行する作業です。データセットを移行するには、メインフレームでダウンロードしたEBCDICデータが必要です。

以下の図は、一般的なデータセットの移行プロセスを示します。

[図 1.3] データセットの移行



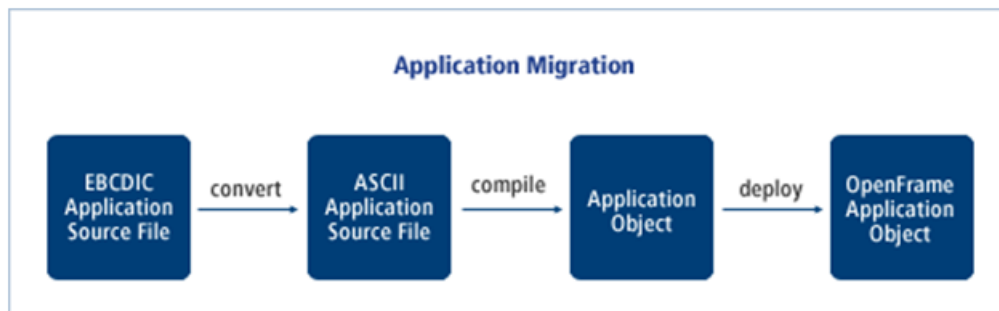
1. データセットのスキーマを含むEBCDICリソースをASCIIリソースに変換します。
2. 変換したASCIIリソースを利用してデータセット変換スキーマを抽出します。
3. データセット変換スキーマを利用して移行対象のEBCDICデータセットを変換およびインポートします。

1.2.3. アプリケーションの移行

アプリケーションの移行は、メインフレームで開発されたアプリケーション・プログラムをOpenFrameで使えるように移行する作業です。アプリケーションを移行するには、メインフレームからダウンロードしたEBCDICソースが必要です。

以下の図は、一般的なアプリケーションの移行プロセスを示します。

[図 1.4] アプリケーションの移行



1. EBCDICアプリケーション・ソースをASCIIアプリケーション・ソースに変換します。
2. 変換されたASCIIアプリケーション・ソースをコンパイルしてランタイム・オブジェクトを作成します。
3. 作成したランタイム・オブジェクトをOpenFrameにデプロイします。

1.2.4. データベースの移行

データベースの移行は、移行後のデータベースの種類に応じて提供されるツールや方法が異なるため、本書ではデータベースの移行に関する説明は省略しています。

参考

詳細については、各データベース・ベンダーのマニュアルを参照してください。

1.3. 移行プロセス

OpenFrameの移行プロセスは、分析と移行に分けられます。

- 分析

プロジェクト・コンサルタントによって行われます。

- 移行

メインフレームのリソースに応じて移行方法とツールが提供されます。詳細については、各章の内容を参照してください。

第2章 リソースの移行

本書では、CICSマップ、CICSリソース、JCLを移行する方法について説明します。

2.1. EBCDICからASCIIへの変換

OpenFrame移行では、EBCDICコードのリソースをASCIIコードのリソースに変換する処理は、すべてのリソースの移行手順で共通して適用されます。

コードの変換は、メインフレームの文字セットとオープン環境の文字セットが異なるために必要であり、移行プロセスで重要な作業です。

本節では、ユーザーが作成したCOBOLソースなどをEBCDICからASCIIにコード変換する方法について説明します。

注

実際の移行では、各リソースに応じてEBCDICからASCIIへの変換方法が異なります。

変換方法

EBCDICデータをASCIIデータに変換する方法はリソースに応じて異なります。非VSAMデータセットは、EBCDICデータをASCIIデータに変換するプロセスが特になく、**dsmigin**ツールによって内部的に変換が実行されます。

dsmiginツールのconvert-onlyオプションを使用すると、EBCDICデータからASCIIデータへの変換のみを行います。また、**dsmigin**ツールの—sourceオプションを指定して、さまざまなリソースのEBCDICデータをASCIIデータに変換することができますが、主にアプリケーションのテキスト・ソース・ファイルのEBCDICデータをASCIIデータに変換する際に使用します。

以下は、ソース・ファイルに使用できる**dsmigin**ツールの例です。SRCFILEを80バイトを1行として認識してASCIIに変換するとき、ソース・ファイルのEBCDIC文字がIBM290を使用していることを考慮して変換します。変換後に不要になったSOSI文字の位置を空白にし、変換された行単位でCONVFILEを作成します。行を区切るときは、改行文字を使用します。

```
$ dsmigin SRCFILE CONVFILE -e JP -l 80 -sosi 2 -C -S -n
```

以下は、上記で使用された各オプションについての説明です。

オプション	説明
-e	変換するソース・データセットの言語を選択します。(JP、US、KR、JAKをサポート)
-l	変換するソース・データセットが固定長レコード・フォーマットの場合、1行のサイズを指定します。
-sosi	変換後のSOSI文字の処理オプションです。
-C	EBCDICデータのASCIIデータへの変換のみを行う場合に使用します。
-S	変換するEBCDICファイルが一般ソースである場合に使用します。(COBOLの場合は、-cobオプションを使用)
-n	変換後、行の区切り文字として改行文字を使用します。

参考

dsmiginツールでサポートするオプションについては、『OpenFrame ツールリファレンスガイド』を参照してください。

以下は、BMSマップ・ファイルを変換する例です。

```
of1a@tmaxs4:of1a]$ dsmigin MSCMAP01 MSCMAP01.map -e JP -l 80 -sosi 2 -S -n
>> M 2019-03-17 17:27:57 [026595:dsmigin_main.c:378] (MIG0001M)
=====< DSMIGIN START >====
source file : MSCMAP01
dataset name: MSCMAP01.map
member name :
unit:
volume:
schema file :
encoding type: JP
record length: 80
record format: L
dsorg:
delimiter:

DSMIGIN: (record count = 158)
>> M 2019-03-17 17:27:58 [026595:dsmigin_main.c:1255] (MIG0002M)
=====< DSMIGIN FINISH >====
```

変換前のコードは以下のとおりです。

<MSCMAP01>

```
. . .
@AÆEOAÆ@×0a~Mđoko÷]k0A0CaE~đokAaaUA~Aa0E×@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```



```

@@@@@@@@@@@@@@@@@AÆEOAÆ@×0a~Mðokoo]k0A0CaE~ðn@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@N0ððð÷@@@@AÆEOAÆ@×0a~Mðokðo]k0A0CaE~ðokAaaUA~Aa0E
×@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@N0ðððð@@@@AÆEOAÆ@×0a~Mðokðo]k0A0C
aE~ookAaaUA~Aa0E×k×a~ø@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@AÆEOAÆ@×0a~
Mðokou]k0A0CaE~ðokAaaUA~MAa0E×kAUa]k@@@@@@@@@@@@@@@@@@@@c@@@@@@@@@@@@@@@@@@@@
@@@@@×a~økE0EaEA0~}B0}@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@N0ð
ððu@@@@AÆEOAÆ@×0a~Mðokoo]k0A0CaE~ðokAaaUA~Aa0E×@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@N0ððnð@@@@AÆEOAÆ@×0a~Mðokoo]k0A0CaE~ookAaaUA~Aa0E×k×a~ø
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@N0ððnn@@@@AÆEOAÆ@×0a~Mðokðo]k0A0CaE~ðokAaaUA~
Aa0E×@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@N0ððno@@@@AÆEOAÆ@×0a~Mðokðo]k0A0CaE
~ookAaaUA~Aa0E×k×a~ø@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@AÆEOAÆ@×0a~
Mðokou]k0A0CaE~ðokAaaUA~MAa0E×kAUa]k@@@@@@@@@@@@@@@@@@@@c@@@@@@@@@@@@@@@@@@@@
@@@@@×a~økE0EaEA0~}B0}@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@N0ð
ðno@@@@AÆEOAÆ@×0a~Mðokoo]k0A0CaE~ðokAaaUA~Aa0E×@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@N0ððno@@@@AÆEOAÆ@×0a~Mðokoo]k0A0CaE~ookAaaUA~Aa0E×k×a~ø
. . .

```

変換後のコードは以下のとおりです。

<MSCMAP01.map>

```

BGLOGO    DFHMSD  TYPE=&SYSPARM,                                X
           LANG=COBOL,                                          X
           MODE=INOUT,                                          X
           CTRL=FREEKB,                                         X
           STORAGE=AUTO,                                        X
           TIOAPFX=YES
*
BGLOGO    DFHMDI  SIZE=(24,80),LINE=1,COLUMN=1
*
           DFHMDF POS=(2,2),LENGTH=37,ATTRB=(NORM,PROT),      X
           HILIGHT=BLINK,                                       X
           INITIAL='***  WELCOME TO OPENFRAME ONLINE  ***'
           DFHMDF POS=(5,3),LENGTH=73,ATTRB=(BRT,PROT),      X
           COLOR=YELLOW,                                        X
           INITIAL='  ***** /***** /*****/**  /**/*****/*X
           /***  /**/**/**/*****'
           DFHMDF POS=(6,3),LENGTH=73,ATTRB=(BRT,PROT),      X
           COLOR=YELLOW,                                        X
           INITIAL='/*****/******/******/**  /**/*****/*X
           *  /*****  /*****/******'
           DFHMDF POS=(7,3),LENGTH=73,ATTRB=(BRT,PROT),      X
           COLOR=YELLOW,                                        X
           INITIAL='/**/**/**/**/**/**/**  /**/**/**/**/**X
           **/**/**/**/**/**/**/**'
           DFHMDF POS=(8,3),LENGTH=73,ATTRB=(BRT,PROT),      X
           COLOR=YELLOW,                                        X

```



```

$ mscadjust -o OUTPUT.adj NA810SM NA810SM.map.atm

[MSCADJUST] input ascii code map : NA810SM.map.atm
[MSCADJUST] input ebcdic code map : NA810SM

* before: [.*.*      ...|D..  ....X  .(...7.....)      *.*      ]

* after: [**      外注店指定  (入力画面)      **      ]

* do you want to change? (Yes:(Enter/Y/y), No:(N/n))Y

-----

* before: [....]

* after: [法人]

* do you want to change? (Yes:(Enter/Y/y), No:(N/n))

-----

* before: [...>]

* after: [店No.]

* do you want to change? (Yes:(Enter/Y/y), No:(N/n))

-----

* before: [ .      .....      .R(..>  ]

* after: [ 区  発注日  伝票No.  ]

* do you want to change? (Yes:(Enter/Y/y), No:(N/n))

-----

* before: [ .....      .....3      ]

* after: [ 取引先  取引先名  ]

* do you want to change? (Yes:(Enter/Y/y), No:(N/n))

$

```

以下は、mscmapcを使用してマップをコンパイルする例です。

参考

1. mscasmc、mscadjustツールについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。
2. マップのコンパイルとデプロイについての詳細は、『OpenFrame OSCマッピングサポートガイド』を参照してください。

CICSが提供するシステム・リソースを使用するCICSアプリケーションが正常に動作するには、必要なリソース定義をCICSシステムに設定する必要があります。また、CICSアプリケーションがOSCシステムに移行された後、正常に動作するには、CICSシステムで使用するリソース定義をOSCシステムに移行する必要があります。

旧バージョンのCICSシステムでは、リソース定義の代わりにリソース管理テーブルを使用する場合があります。リソース管理テーブルは、OSCシステムでサポートできるリソース定義に変換するための追加プロセスが必要です。OSCシステムでは、旧バージョンのリソース管理テーブルのDCT(Destination Control Table)とJCT(Journal Control Table)をTDQUEUEとJOURNALMODELに変換するツールを提供しています。

```
oscdct2rd -q OSCGROUP DCT.dat TDQUEUE.dat
```

以下は、oscjct2rdツールを使用してJCTをJOURNALMODELリソース定義に移行する例です。

```
oscjct2rd -g OSCGROUP JCT.dat JOURNALMODEL.dat
```

リソース定義には、OSCシステムでサポートしないか、使用しないフィールドが含まれていることがあります。このようなフィールドは、**oscsdgen**ツールを使用してリソースを登録する際に無視されます。

以下は、resources.datファイルに保存されているリソース定義をOSC00001リージョンで使用中のOSC SDデータセットに登録する例です。

```
$ oscsdgen -r OSC00001 resources.dat
```

参考

1. oscdct2rd、oscjct2rd、oscsdgenツールについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。
 2. OSCでサポートするリソースのフィールド属性については、『OpenFrame OSCリソース定義ガイド』を参照してください。
-

2.4. JCLの移行

JCLの移行は、既存のメインフレームのJES環境で実行されていたJCLをダウンロードして、EBCDICコードのJCLソース・ファイルをASCIIコードに変換する処理です。これにより、既存のJCLソースを修正することなく、OpenFrameのTJES環境で実行することができます。

ただし、サード・パーティ・ユーティリティを使用する場合は、メインフレームとUNIXシステムの違いにより、JCLのSYSINまたはパラメータの送信方法が異なるため、サード・パーティ・ユーティリティに合わせてJCLを変換する作業が必要です。また、メインフレームで使用していたユーティリティをOpenFrameで使用できない場合があるため、OpenFrameで同じ機能を提供するユーティリティに変換する必要があります。

本節では、最も多く使用されているDB2ユーティリティの移行について説明します。

2.4.1. DB2

メインフレームのホストであるDB2は、UNLOAD(DB2に保存されているデータを直列化してファイルに書き出すこと)を実行した後、制御ファイルを作成します。制御ファイルは、UNLOAD処理でダウンロードしたデータやファイルを再びDB2にLOAD(抽出したファイルを復元すること)するとき使用するファイルです。

UNIXシステムで実行されるDB2 HPUもメインフレームと同様にUNLOADの処理後、LOADを実行するときに必要な制御ファイルを作成します。ただし、メインフレームとUNIXは制御ファイルの文法的な違いがあるため、メインフレームで作成された制御ファイルをUNIXシステムのDB2 HPUでは使用できません。そのため、LOAD JCLのSYSINにインストリームで指定されたLOAD制御文を、ASC

FROMAT METHODLに変換した後、その制御ファイルをUNIXシステムのDB2 HPUの文法に合わせて変換します。

以下は、メインフレームのDB2 LOAD制御文をUNIX UDB LOAD制御文に変換する例です。

変換前

```
LOAD DATA RESUME YES LOG NO INDON SYSREC00 INTO TABLE
  LNID2LC0.TLCP0
(
  CIJUN_YM              POSITION(      1      )
  CHAR(                6) ,
  ORG_CD                POSITION(      7      )
  CHAR(                7) ,
  SNPM_SORT_GB          POSITION(     14      )
  CHAR(                1) ,
  YUJICHASU             POSITION(     15      )
  CHAR(                3) ,
  DSBHR_10              POSITION(    18:    25)
  DECIMAL              ,
  YJBHR_10              POSITION(    26:    33)
  DECIMAL              ,
  DSBHR_30              POSITION(    34:    41)
  DECIMAL              ,
  YJBHR_30              POSITION(    42:    49)
  DECIMAL              ,
  DSBHR_60              POSITION(    50:    57)
  DECIMAL              ,
  YJBHR_60              POSITION(    58:    65)
  DECIMAL              ,
  DSBHR_90              POSITION(    66:    73)
  DECIMAL              ,
  YJBHR_90              POSITION(    74:    81)
  DECIMAL              )
```

変換後

```
LOAD FROM $SYSREC00      OF ASC
MODIFIED BY STRIPTBLANKS NULLINDCHAR=?
BINARYNUMERICS PACKEDDECIMAL CODEPAGE=970 RECLLEN=81
METHOD L (
1 6,
7 13,
14 14,
15 17,
18 25,
26 33,
34 41,
42 49,
```

```
50 57,  
58 65,  
66 73,  
74 81)  
NULL INDICATORS ( 0,0,0,0,0,0,0,0,0,0,0,0 )  
INSERT INTO LNID2LC0.TLCP0  
GIJUN_YM,  
ORG_CD,  
SNPM_SORT_GB,  
YUMICHASU,  
DSBHR_10,  
YJBHR_10,  
DSBHR_30,  
YJBHR_30,  
DSBHR_60  
YJBHR_60  
DSBHR_90  
YJBHR_90)
```


第3章 データセットの移行

本章では、データセットを移行する方法について説明します。

3.1. 概要

データセットは、データ・レコードの論理的な集合体です。またレコードは、アプリケーションで使用される情報の基本単位です。データセットは大きく非VSAMデータセットとVSAMデータセットに分けられます。

参考

データセットについての詳細は、『OpenFrame データセットガイド』を参照してください。

- 非VSAMデータセット

以下は、非VSAMデータセットのPDS、GDG、SAMデータセット、ISAMデータセットの移行方法です。

区分	説明
PDS	PDSは、ディレクトリとメンバーから構成されるデータセットです。PDSは、 pdsgen または dscreate ツール使用して作成し、PDSの各メンバー・ファイルは、 dsmigin ツールを使用して文字セットを変換します。
GDG	GDGは、世代で構成されるデータセットのグループです。GDGは、 gdgcreate ツールを使用してカタログに登録し、SAMデータセットの移行方法に従って移行します。
SAMデータセット	一般的な非VSAMデータセットであるSAMデータセットは、COBOLコピーブックなどを参照してデータセット変換スキーマを作成した後、 dsmigin ツールを使用してOpenFrameデータセットにインポートします。
ISAMデータセット	ISAMデータセットはOpenFrameでサポートしており、VSAM KSDSに代替して使用することもできます。

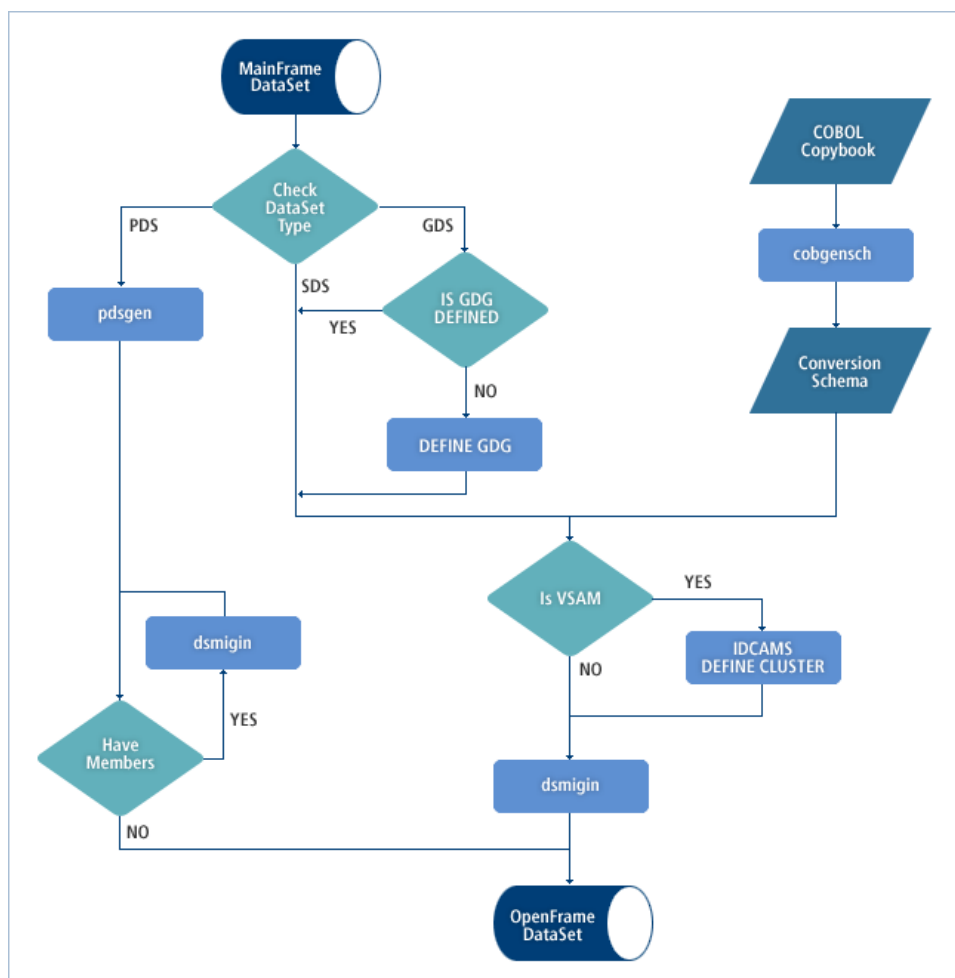
- VSAMデータセット

以下は、VSAMデータセットのKSDS、ESDS、RRDS、LDSの移行方法です。

区分	説明
KSDS	KSDSは、 idcams ツールのDEFINE CLUSTERコマンドでOpenFrame VSAMデータセットを定義します。また、 dsmigin ツールのRECATALOGオプションを使用してデータをVSAMデータセットにインポートします。
ESDS	KSDSの移行方法と同じです。
RRDS	RRDSは、各レコードごとに番号が付与されます。データセットの中に番号が欠けているレコードがなければ、VSAM KSDSと同じ方法で移行できます。ただし、番号が欠けているレコードがある場合は、別のRRDSインポート・プログラムを作成して移行する必要があります。
LDS	現在VSAM LDSは、OpenFrameでサポートしていません。VSAM LDSのサポートが必要な場合は、弊社の技術サポート・チームにお問い合わせください。

メインフレームのデータセットをOpenFrameに移行する方法は、次のようにデータセットの種類によって異なります。

[図 3.1] データセットの移行プロセス



3.2. 非VSAMデータセットの移行

OpenFrameで一般的な非VSAMデータセットの移行プロセスは以下のとおりです。

1. 移行対象の確認およびソース・ファイルのダウンロード
2. データセット・レイアウトの分析
3. データセット変換スキーマの作成
4. ターゲット・データセットの移行
5. PDSデータセットの移行
6. GDGベースの定義
7. 可変長データセットの移行
8. データセット移行の検証

3.2.1. 移行対象の確認およびソース・ファイルのダウンロード

移行対象となるデータセットの一覧を作成することは、移行プロジェクトを成功させるための重要な作業です。JCLまたはプロシージャを分析して一覧を作成することもできますが、大規模なプロジェクトでは、OpenFrameへの移行によって不要となったり、SMSポリシーやセキュリティ・ポリシーによってデータセット名が変更される場合もあるため、お客さまから正確な一覧を取得する必要があります。

移行するデータセットが決まったら、ホスト(運用中のメインフレーム)で使用しているデータセットのソース・ファイルをダウンロードします。ただし、データセットをASCII形式でダウンロードすると、特殊文字、SOSI、2バイト文字が正常に変換されず問題が発生するため、ホストで使用しているEBCDIC形式のデータセットをそのままダウンロードしてください。

3.2.2. データセット・レイアウトの分析

データセットのレイアウト分析は、データセットの移行を正常に実行するために最も重要なステップです。現在使用中のデータセットは、COBOLコピーブックやデータベース・スキーマからデータセットのレイアウト情報を抽出できますが、長期間使用していないデータセットは、直接確認しないと、レイアウトを正確に分析することは難しいです。したがって、顧客会社からレイアウト情報を取得し、データセットの移行を行うのが一般的です。

通常は、1つのデータセットのすべてのレコードは、1つのレイアウトが適用されますが、一部のデータセットは、各レコードに異なるレイアウトが適用される場合もあります。これは、COBOLのREDEFINES文によって複数のレイアウトが生成された場合や、データセットのレコードが可変長である場合に発生します。

3.2.3. データセット変換スキーマの作成

データセットのレイアウト情報は、運用の利便性を考慮してさまざまな形式で提供されます。これらをすべてサポートすることは難しいため、以下のような標準のCOBOLコピーブックまたはPL/Iインクルードを変換して使用します。

<XTBC106.cpy>

```
01 I1.
   05 KYAKUMEI-KN          PIC X(0018).
   05 BTN-CD               PIC X(0004).
   05 KYAKU-NO             PIC X(0007).
   05 ATUKAI-CD            PIC X(0003).
   05 MDY-CD               PIC X(0003).
   05 YAKU-YMD             PIC S9(0009) COMP-3.
   05 UKEW-YMD             PIC S9(0009) COMP-3.
   05 KOYU-MEI-CD          PIC S9(0005) COMP-3.
   05 KAISU                PIC S9(0005) COMP-3.
   05 GO .
      10 GO-1               PIC X(0001).
      10 GO-2               PIC X(0001).
      10 GO-3               PIC X(0001).
```

データセットのレイアウト情報を示すCOBOLコピーブック・ファイル<XTBC106.cpy>がある場合は、**cobgensch**ツールを使用してデータセット変換スキーマ・ファイルを作成することができます。

以下のコマンドでは、**-r**オプションを使用してレコード長を54に指定しています。このようにレコード長を指定する場合は、COBOLコピーブック・ファイルの各フィールドの長さを足した実際のレコード長と一致するかを確認する必要があります。

```
$ cobgensch XTBC106.cpy -r 54
```

以下は、上記のサンプル・ファイル<XTBC106.cpy>を**cobgensch**ツールを実行して作成されたデータセット変換スキーマ・ファイルの内容です。

<XTBC106.conv>

```
* Schema Version 7.1
L1, 01, I1, NULL, NULL, 0, 1:1,
L2, 05, KYAKUMEI-KN, EBC_ASC, NULL, 18, 1:1,
L3, 05, BTN-CD, EBC_ASC, NULL, 4, 1:1,
L4, 05, KYAKU-NO, EBC_ASC, NULL, 7, 1:1,
L5, 05, ATUKAI-CD, EBC_ASC, NULL, 3, 1:1,
L6, 05, MDY-CD, EBC_ASC, NULL, 3, 1:1,
L7, 05, YAKU-YMD, PACKED, NULL, 5, 1:1,
L8, 05, UKEW-YMD, PACKED, NULL, 5, 1:1,
L9, 05, KOYU-MEI-CD, PACKED, NULL, 3, 1:1,
L10, 05, KAISU, PACKED, NULL, 3, 1:1,
L11, 05, GO, NULL, NULL, 0, 1:1,
L12, 10, GO-1, EBC_ASC, NULL, 1, 1:1,
```

```

L13, 10, GO-2, EBC_ASC, NULL, 1, 1:1,
L14, 10, GO-3, EBC_ASC, NULL, 1, 1:1,

* Condition
L0, "\0", ( L1 L2 L3 L4 L5 L6 L7 L8 L9 L10 L11 L12 L13 L14 )

```

COBOLコピーブックのPIC句で構成された各構文がデータセット変換スキーマ形式に変換されていることが確認できます。PIC X句はEBCDIC文字をASCII文字に変換するためのEBC_ASCタイプに指定され、PIC S9のCOMP-3句はPACKEDタイプに指定されていることが分かります。ここで使用されたタイプの他に、ゾーン10進数値を表現するZONEDタイプと2バイト文字を表現するGRAPHICタイプがあります。

上記で説明したCOBOLコピーブック・ファイル以外に、以下のようなPL/Iインクルード形式のデータセット・レイアウト情報を変換して使用することもできます。

<XTBC107.inc>

```

3  ID CHAR(02),
3  NAME CHAR(05),
3  CODE,
   5  CODE_NUM    PIC'(03)9',
   5  CODE_DAT    PIC'(03)X',
3  ETC FIXED DEC (3, 0) ;

```

データセットのレイアウト情報を表すPL/Iインクルード・ファイル<XTBC107.inc>がある場合は、以下のように**pligensch**ツールを使用してデータセット変換スキーマ・ファイルを作成することができます。

以下のように、-sオプションを指定する場合は、PL/Iインクルード・ファイルの一部だけを変換します。

```
$ pligensch -s XTBC107.inc
```

以下は、サンプル・ファイル<XTBC107.inc>を**pligensch**ツールで実行して作成したデータセット変換スキーマ・ファイルの内容です。PL/Iインクルード・ファイルで各フィールド・タイプを示すCHAR句、PIC 9句、FIXED DEC句が、以下のようにEBC_ASC、ZONED、PACKEDタイプにそれぞれ変換されていることを確認できます。

<XTBC107.conv>

```

* Schema Version 5.1
L1, 01, ID_STRUCTURE, NULL, 0, 1,
L2, 03, ID, EBC_ASC, 2, 1,
L3, 03, NAME, EBC_ASC, 5, 1,
L4, 03, CODE, NULL, 0, 1,
L5, 05, CODE_NUM, ZONED, 3, 1,
L6, 05, CODE_DAT, EBC_ASC, 3, 1,
L7, 03, ETC, PACKED, 2, 1,

```

```
* Condition
L0, "\0", ( L1 L2 L3 L4 L5 L6 L7 )
```

参考

スキーマ・ファイルの構造についての詳細は、本書の「[付録 C. スキーマ・ファイルの構造](#)」を参照してください。

3.2.4. ターゲット・データセットの移行

移行対象のソース・データセットと変換スキーマ・ファイルが準備されたら、**dsmigin**ツールを使用してデータセットを移行します。

以下は、ソース・ファイル<XTB.T66.raw>と変換スキーマ・ファイル<XTB.T66.conv>を基に、**dsmigin**ツールを使用してデータセット<XTB.T66>を作成する例です。

```
$ dsmigin XTB.T66.raw XTB.T66 -s XTB.T66.conv -e JP -l 800
```

dsmiginツールは、各レコード・データのコード変換およびデータセットのカatalog登録を行います。

参考

dsmiginツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

3.2.5. PDSの移行

PDSは、ディレクトリとメンバー・ファイルから構成されるデータセットで、JCLまたはPROCのDSNパラメータにPDS名と括弧で囲まれたメンバー名を一緒に表記します。

```
//SYSIN DD DSN=PLNI.EDVR.SYSIN(LNIYY01),
// DISP=(SHR,PASS,KEEP)
```

PDSは通常、実行モジュールを管理するライブラリ・データセットとして使用されるか、あるいはJCL、PROC、SYSINなどのファイルを管理するライブラリ・データセットとして使用されます。

PDSはOpenFrameでUNIXファイル・システムのディレクトリとして管理され、**pdsngen**ツールまたは**dscreate**ツールで作成することができます。

以下は、**dscreate**ツールを使用してTEST.PDSという名前のPDSを作成する例です。

```
$ dscreate -o P0 TEST.PDS
```

PDSを作成する際に注意すべきことは、SYSINタイプのデータセットは、レコード・フォーマットを「L」タイプに指定する必要があります。「L」タイプは、メインフレームには存在しないレコード・フォーマットで、UNIXプラットフォームに合わせて改行(linefeed)を1つのレコード単位として認識するために、OpenFrameでサポートする新しいレコード・フォーマットです。

実行モジュールを管理するPDSには、コンパイルされた実行モジュールをPDSのディレクトリ・メンバーとして保存します。その他に、JCL、PROC、SYSINなどのファイル管理のために使用されるPDSには、メインフレームからダウンロードしたEBCDIC形式のソース・ファイルを**dsmigin**ツールで文字セットを変換した変換ファイルをPDSのメンバーとして保存します。

3.2.6. GDGベースの定義

GDGベースはカタログ項目であり、**gdgcreate**ツールで作成することができます。GDGベースを作成する際にGDG LIMIT値は、お客さまから正確な情報の提供を受けて作成してください。

以下は、**gdgcreate**ツールを使用してGDGベースを定義するJCLの例です。

```
$ gdgcreate -l 255 TEST.GDG01
```

GDGベースが特定のユーザー・カタログに登録されると、関連するすべてのGDS（GDGグループ内のデータセット）データセットも同ユーザー・カタログに登録されます。GDGベースを作成せずにGDSデータセットのカタロギングを行うと、AMS_ERR_GDG_NOT_REGISTERED (-4025)エラーが発生します。

3.2.7. 可変長データセットの移行

可変長データセットの移行は、一般的なデータセットの移行プロセスと同じです。ただし、データセットのレイアウトを分析して変換スキーマを作成する作業が複雑なので注意が必要です。

メインフレームからダウンロードした可変長データセットのソース・データセットは、下図のように順次ファイルと同一形式です。ただし、各レコードは4バイトのRDW(Record Descriptor Word)と引き続くデータで構成されます。RDWの最初の2バイトは、4バイトのRDWを含むレコード長情報を持ち、残りの2バイトは0で構成されます。このRDWのレコード情報で、レコードの先頭と末尾を区分的ことができます。

[図 3.2] 可変長データセットのソース・ファイル

```

00000000: 009a 0000 3034 3036 3539 3031 3230 3030 ....040659012000
00000010: 3330 3031 3135 3336 3738 3839 3930 3632 3001153678899062
00000020: 3939 3930 3730 3135 3030 3030 3030 3030 9990701500000000
00000030: 3030 3030 3030 3120 2020 2020 2020 2020 00000001
00000040: 2020 2020 2020 2020 2020 2020 2020 2020
00000050: 2020 2020 2020 2020 2020 2020 2020 2020
00000060: 2020 2020 2020 2020 2020 2020 2020 2031 1
00000070: 3030 3736 3130 3030 3030 3030 3030 3030 0076100000000000
00000080: 3030 3030 3032 3935 3539 3830 3939 3036 0000029559809906
00000090: 3039 3939 3036 3039 016d 009a 0000 3034 09990609.m....04
00000a00: 3036 3539 3031 3230 3030 3330 3031 3135 0659012000300115
00000b00: 3432 3937 3639 3930 3632 3939 3930 3730 4297699062999070
00000c00: 3135 3030 3030 3030 3030 3030 3030 3030 1500000000000000

```

固定長データセットと同様に可変長データセットも、データセットのレイアウトをCOBOLコピーブック形式にまとめたものを**cobgensch**ツールを実行して変換スキーマを作成します。しかし、プロジェクトの状況に応じて、COBOLコピーブック形式にまとめる段階を省略し、下記のような変換スキーマ・ファイルを早速作成して使用する場合があります。

```

* Schema Version 7.1
L1, 01, V-REC, NULL, NULL, 0, 1:1,
L2, 05, REC-KEY, EBC_ASC, NULL, 8, 1:1,
L3, 05, ODO-LEN, COPY, NULL, 2, 1:1,
L4, 05, ODO-FLD, NULL, NULL, 0, 0:4000, ODO-LEN
L5, 07, ODO-DAT, EBC_ASC, NULL, 1, 1:1,

* Condition
L0, "\0", ( L1 L2 L3 L4 L5 )

```

上記例のスキーマ・ファイルは、ODO-LENフィールドの値によってODO-FLDの長さが変わる可変長データセットの例です。各レコードの変換を行う際に、レコードのODO-LENフィールド値を読み込んで、ODO-FLDの繰り返し回数とデータセットの長さを決定します。ソース・データセットと変換スキーマ・ファイルの準備ができれば、固定長データセットと同様に、**dsmigin**ツールを使用してデータセットを移行します。

以下は、ソース・ファイル<TEST.VAR.raw>と変換スキーマ・ファイル<TEST.VAR.conv>を基に、**dsmigin**ツールを使用してデータセット<TEST.VAR>を作成する例です。

```
$ dsmigin TEST.VAR.raw TEST.VAR -s TEST.VAR.conv -e JP -f VB
```

3.2.8. データセット移行の検証

ターゲット・データセットの移行が完了したら、**listcat**ツールを使用してデータセットがカタログに正常に登録されているか確認します。その後、**dsvview**ツールで移行されたデータセットのデータを検索して、ターゲット・データセットに正しく移行されているかを検証します。

以下は、**dsvview**ツールを使用してデータセットを検索した画面です。

[図 3.3] devviewツールを使用したデータセットの検索

```
00000001: I 2 .. 400J2010S0J1.. ..l ..l 2 ....P..3.. 1 1.....*.....
00000002: I 2 .. 400J2010S0J2.P. ..< ..l 2 .....c..#.l 1.....猶.....
00000003: I 2 .. 400J2010S0J4.< ..l ..l 2 .....3.. 1 1.....*.....h..
00000004: I 2 .. 400J2010S0| .....< Command > 1 1.....@.....@..
00000005: I 2 .. 400J2010S0| PAGE UP CTRL+b 1 2.....%.0.....u..
00000006: I 2 .. 400J2010S0| PAGE DOWN CTRL+f 1 2.....%.0.....u..
00000007: I 2 .. 400J2010S0| HOME LINE ~ 1 1.....*.....
00000008: I 2 .. 400J2010S0| END LINE $ 1 1.....*.....
00000009: I 2 .. 400J2010S0| GO to 125 125G 1 1.....*.....
00000010: I 2 .. 400J2010S0| GO to 125 :125 1 2.....%.0.....u..
00000011: I 2 .. 400J2010S0| GO to END G 1 1.....*.....
00000012: I 2 .. 400J2010S0| GO to up/down 5 5k/5j 1 2.....%.0.....u..
00000013: I 2 .. 400J2010S0| GO to left/right 5 5h/5l 1 2.....%.0.....u..
00000014: I 2 .. 400J2010S0| FIND abc /abc 1 2.....%.0.....u..
00000015: I 2 .. 400J2010S0| FIND abc at 5 col /abc 5 1 2.....%.0.....u..
00000016: I 2 .. 400J2010S0| FIND for HEXA abc //816283 1 2.....%.0.....u..
00000017: I 2 .. 400J2010S0| FIND NEXT n 1 2.....%.0.....u..
00000018: I 2 .. 400J2010S0| Refresh Screen CTRL+l 1 2.....%.0.....u..
00000019: I 2 .. 400J2010S0| HELP :h 1 2.....%.0.....u..
00000020: I 2 .. 400J2010S0| QUIT :q 1 1.....*.....
00000021: I 2 .. 400J2010S0| Press any key.. 1 1.....*.....
00000022: I 2 .. 400J2010S0| ..... 1 2.....%.0.....u..
00000023: I 2 .. 400J2010S0J8.'l ..% ..l ..... 1 2.....%.0.....u..
00000024: I 2 .. 400J2010S0J8.'l ..% ..l ..... 1 2.....%.0.....u..
00000025: I 2 .. 400J2010S0J8.'l ..% ..l ..... 1 2.....%.0.....u..
---END---
```

3.3. VSAMデータセットの移行

OpenFrameの一般的なVSAMデータセットの移行プロセスは以下のとおりです。

1. **idcams**ツールのDEFINE CLUSTERコマンドで空のVSAMデータセットを定義します。
2. **dsmigin**ツールのRECATALOGオプションでデータをVSAMデータセットにインポートします。

上記の方法で移行されないデータセット(データセットの中に番号が欠けているレコードがある場合)は、別のRRDSインポート・プログラムを作成して移行する必要があります。

3.3.1. VSAMデータセットの定義

VSAMデータセットは、**idcams**ツールのDEFINE CLUSTERコマンドで定義します。

以下は、**idcams**ツールを使用してVSAMデータセットを定義する例です。

```
$ idcams DEFINE CLUSTER -n TEST.KSDS -o KS -k 6,0 -l 250,250
```

参考

1. OpenFrameでは、データをコピーブック・フィールドごとに変換して管理できるようにVSAMデータセットを定義する機能(TSAM RDB)を提供しています。詳細については、[「付録 D. TSAM RDB」](#)を参照してください。
2. **idcams**ツールを使用してVSAMデータセットを定義する方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

3.3.2. VSAMデータセットのインポート

VSAMデータセットを定義した後、**dsmigin**ツールの-R(Recatalog)オプションでデータをVSAMデータセットにインポートします。

以下は、**dsmigin**ツールを使用して、<XTB.XY1315>という非VSAMデータセットの全レコード・データをVSAMデータセット<TEST.VSAM.KSDS>にインポートする例です。

```
$ dsmigin XTB.XY1315 TEST.KSDS -s XTB.XY1315.conv -e JP -R
```

OpenFrameの以前バージョンでは、移行ツールを使用してVSAMデータセットをインポートする方法をサポートしていなかったため、**IDCAMS**ユーティリティのREPROコマンドを使って、SAM形式の非VSAMデータセットをVSAMデータセットにコピーする方法を使用しました。しかし、現バージョンでは、**dsmigin**ツールを使用してVSAMデータセットをインポートする方法をサポートするようになったので、**IDCAMS**ユーティリティよりは**dsmigin**ツールを使用することをお勧めします。

3.3.3. データセット・インポート・プログラムの作成

データセットの中に番号が欠けているレコードが存在するVSAM RRDSのように、**dsmigin**ツールを使用してインポートできないデータセットは、別のインポート・プログラムを作成してインポートします。

インポート・プログラムを使用したデータセットの移行プロセスは以下のとおりです。

1. メインフレームからダウンロードしたソース・データセットの文字セットを、**dsmigin**ツールを使用して変換します。
2. **idcams**ツールのDEFINE CLUSTERコマンドを使用して、空のVSAMデータセットを定義します。
3. データセット・インポート・プログラムを実行してデータセットをインポートします。

データセット・インポート・プログラムは、なるべくUNIXプラットフォームで広く使われているC言語で作成してください。

参考

WRITE関数を呼び出して個々のレコードを正しいキーの位置にインポートするプログラムを作成する方法については、『OpenFrameデータセットガイド』の「付録C. データセットI/O API」を参照してください。

3.4. 静的変換と動の変換

OpenFrameのデータセットの移行方法には、静的変換と動の変換の2つの処理方法があります。データセットを移行する際に、データセットのスキーマ情報を確認して、データセットを静的変換で移行するか動の変換で移行するかを決定します。

3.4.1. 静的変換と動変換の比較

- 静的変換

静的変換は、データセットの移行を実行する際、ユーザーが最初に\$\$COND条件に指定した移行パスから1つを決定し、すべてのレコードをこのパス情報を使用して移行する方法です。

- 動変換

動変換は、データセットの移行時に「動変換テーブル」を管理しながら、ユーザーが指定したCOND条件をすべてのグループ・アイテムに対してチェックし、その情報を動変換テーブルに適用して、COND条件の分析が完了した後にその情報を使用して移行する方法です。

動変換はすべてのグループ・アイテムごとに\$\$COND条件を判断するため、静的変換よりパフォーマンスが低下します。ただし、「[3.4.2. 動変換のみ使用できる場合](#)」のように静的変換を使用できない場合や、「[3.4.3. 動変換の使用を推奨する場合](#)」のように静的変換は使用できるが、動変換のほうがスキーマ情報を簡単に作成できる場合もあります。その他の場合は、必要に応じて動変換を使用することができます。

3.4.2. 動変換のみ使用できる場合

以下の場合、動変換を指定する必要があります。

- スキーマ・ファイルを作成するコピーブックの\$\$COND条件を判断するフィールドがOCCURS内にある場合

\$\$COND条件を判断するフィールドがOCCURS内にある場合に静的変換を使用すると、最初のOCCURSの値によって変換する移行パスを決定しますが、最初のOCCURS以降の条件を判断するフィールドの値が最初のOCCURS値と異なる場合も最初に指定した移行パスで変換処理を行うため、正常に変換処理が行われないことがあります。

```
01 A OCCURS 10 TIMES.  
    05 KBN PIC X.  
    05 B1 PIC N(3)  
    05 B2 PIC X(6) REDEFINES B1.  
  
$$COND : KBN : "1" : B1  
$$COND : KBN : !"1" : B2
```

3.4.3. 動変換の使用を推奨する場合

以下の場合には、静的変換を使用することもできますが、動変換を使用すると、より効率的にスキーマ・ファイルを作成することができます。

- 条件を判断する項目による変換対象項目がLIST形式で存在する場合

\$\$COND条件を満たし、変換対象項目がLIST形式で存在する場合、静的変換を使用すると、すべての項目を記述しなければなりません。理論的には最低 2^n 個を作成する必要があります。ただし、動変換を使用すると、この数を 2^n までに減らすことができます。

```
03 A OCCURS 10 TIMES.  
    05 KBN1 PIC X.  
    05 B1 PIC N(3).  
    05 B2 PIC X(6) REDEFINES B1.  
    05 C1 PIC X(5).  
    05 C2 PIC 9(5) REDEFINES C1.  
    05 D1 PIC X(5).  
    05 D2 PIC 9(5) REDEFINES D1
```

<静的変換を使用した\$\$COND条件の記述>

```
$COND : KBN1 : X"01" : B1, C1, D1  
$COND : KBN1 : X"02" : B1, C1, D2  
$COND : KBN1 : X"03" : B1, C2, D1  
$COND : KBN1 : X"04" : B1, C2, D2  
$COND : KBN1 : X"05" : B2, C1, D1  
$COND : KBN1 : X"06" : B2, C1, D2  
$COND : KBN1 : X"07" : B2, C2, D1  
$COND : KBN1 : X"08" : B2, C2, D2
```

<動変換を使用した\$\$COND条件の記述>

```
$COND : KBN1 : X"01", X"02", X"03", X"04" : B1  
$COND : KBN1 : X"04", X"05", X"06", X"07" : B2  
$COND : KBN1 : X"01", X"02", X"05", X"06" : C1  
$COND : KBN1 : X"03", X"04", X"07", X"08" : C2  
$COND : KBN1 : X"01", X"03", X"05", X"07" : D1  
$COND : KBN1 : X"02", X"04", X"06", X"08" : D2
```

3.4.4. 動変換の使用手法と処理順序

データセットを動変換で移行するには、**cobgensch**ツールの-Dオプションを指定して動変換スキーマを作成します。

```
$ cobgensch -D XXX.scm
```

動変換スキーマを作成したら、**dsmigin**ツールを使用して動変換マイグレーションを実行します。特にオプションを指定しなくても、作成されたスキーマ情報を使用して変換します。

```
$ dsmigin EBCDIC ASCII -s XXX.scm
```

参考

cobgenschツールとdsmiginツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

3.5. データセットの移行

本節では、固定長データセット、可変長データセット、REDEFINES文が使用されるデータセットの移行について説明します。

3.5.1. 固定長データセット

以下は、レコード長が16で、3つのレコードを持つメインフレームEBCDICデータセット<FL.SAMPLE.raw>です。

[図 3.4] 固定長データセットのソース・ファイル

```
00000000: f0f0 f0f1 c5c2 c3c1 e2c3 c1c1 c1f1 f1c1 .....  
00000010: f0f0 f0f2 c5c2 c3c1 e2c3 c2c2 c2f2 f2c2 .....  
00000020: f0f0 f0f3 c5c2 c3c1 e2c3 c3c3 c3f3 f3c3 .....
```

以下のCOBOLコピーブック<FL.SAMPLE.cpy>の例から、データセットのレイアウト情報を確認できます。

<FL.SAMPLE.cpy>

```
01 FIXEDLENGTH.  
    05 KEY          PIC X(4).  
    05 TYPE          PIC X(6).  
    05 DATA.  
        10 EBCDATA    PIC X(3).  
        10 ZONEDDATA  PIC S9(3).
```

データセットのレイアウト情報を確認したら、**cobgensch**ツールを使用して以下のように変換スキーマ・ファイルを作成します。

```
$ cobgensch FL.SAMPLE.cpy
```

以下は、**cobgensch**ツールの実行によって作成された変換スキーマ・ファイルです。

<FL.SAMPLE.conv>

```
* Schema Version 7.1  
L1, 01, FIXEDLENGTH, NULL, NULL, 0, 1:1,  
L2, 05, KEY, EBC_ASC, NULL, 4, 1:1,  
L3, 05, TYPE, EBC_ASC, NULL, 6, 1:1,  
L4, 05, DATA, NULL, NULL, 0, 1:1,  
L5, 10, EBCDATA, EBC_ASC, NULL, 3, 1:1,
```

```
L6, 10, ZONEDDATA, ZONED, TRAILING, 3, 1:1,
```

```
* Condition
```

```
L0, "\0", ( L1 L2 L3 L4 L5 L6 )
```

ソース・データセットと変換スキーマ・ファイルを準備できたら、**dsmigin**ツールを使用してデータセットを移行します。

```
$ dsmigin FL.SAMPLE.raw FL.SAMPLE.TEST -s FL.SAMPLE.conv -l 16
```

データセットの移行が完了したら、**dsvview**ツールを使用して結果を確認します。

[図 3.5] 固定長データセットの移行結果

```
00000001: 001EBCASCAA111
00000002: 002EBCASCB222
00000003: 003EBCASCCC333
-----END-----
FL.SAMPLE.TEST          12. 175          3 lines
Hex: 00303031454243415343414141313131
```

3.5.2. 可変長データセット

以下は、可変長レコードを持つメインフレームのデータセットの例です。

```
00000001016AAAAA
00000002013C3
00000003021CCCCCCCCC
```

メインフレームからダウンロードした可変長データセット<VB.SAMPLE.raw>は、以下のように各レコードの先頭に4バイトのRDW(Record Description Word)が含まれています。

[図 3.6] 可変長データセットのソース・ファイル

```
0000000: 001a 0000 f0f0 f0f0 f0f0 f0f1 000c c1c1 .....
0000010: c1c1 c1c1 c1c1 c1c1 0012 0000 f0f0 .....
0000020: f0f0 f0f0 f0f2 0004 c2c2 c2c2 0023 0000 .....#..
0000030: f0f0 f0f0 f0f0 f0f3 0015 c3c3 c3c3 c3c3 .....
0000040: c3c3 c3c3 c3c3 c3c3 c3c3 c3c3 c3c3 c3 .....

```

このデータセットのレイアウト情報は、コピーブックに以下のように表示されます。

<VB.SAMPLE.cpy>

```
01 V-REC.
      05 REC-KEY      PIC X(8).
      05 ODO-LEN      PIC S9(4) COMP.
      05 ODO-FLD      OCCURS 4000 TIMES
                        DEPENDING ON ODO-LEN.
      07 ODO-DAT      PIC X(1).
```

指定されたコピーブックを入力として**cobgensch**ツールを実行すると、以下のような移行スキーマ・ファイルが作成されます。

```
* Schema Version 7.1
L1, 01, V-REC, NULL, NULL, 0, 1:1,
L2, 05, REC-KEY, EBC_ASC, NULL, 8, 1:1,
L3, 05, ODO-LEN, COPY, NULL, 2, 1:1,
L4, 05, ODO-FLD, NULL, NULL, 0, 1:4000, ODO-LEN
L5, 07, ODO-DAT, EBC_ASC, NULL, 1, 1:1,

* Condition
L0, "\0", ( L1 L2 L3 L4 L5 )
```

ソース・データセットと移行スキーマ・ファイルが準備されたら、**dsmigin**ツールを使用してデータセットを移行します。

```
$ dsmigin VB.SAMPLE.raw VB.SAMPLE.TEST -s VB.SAMPLE.conv -e JP -f VB
```

<VB.SAMPLE.raw>はダウンロードしたソース・ファイルであり、<VB.SAMPLE.TEST>は移行後に保存されるデータセットです。また、<VB.SAMPLE.conv>は移行スキーマ・ファイルです。

3.5.3. REDEFINES文が使用されたデータセット

以下のように、REDEFINES文が含まれているレイアウトを持つデータセットは、分岐条件に関する情報が必要です。

```
01 A. 30.
    05 B      PIC X(01).
    05 C      PIC X(05).
    05 D      REDEFINES  C.
    10 E      PIC X(02).
    10 F      PIC X(03).
```

フィールドBの値がAの場合にのみ再定義されたDの情報を参照し、その他の場合は、Cの情報を参照するには、コピーブックに\$\$COND条件文を追加する必要があります。

以下は、\$\$COND文の使用方法です。

```
$$COND : NAME_01 : VALUE_01 [: NAME_02 : VALUE_02 ... ] : REDEFINE_NAME [...]
```

- \$\$COND

- 条件文の開始を示します。

- NAME_01 : VALUE_01

- フィールド名とそのフィールドに該当する条件値を順次に入力します。各項目を区切るときは、コロン(:)を使用します。dsmiginツールを使用して移行する場合、VALUE_#の条件値はすでに定義されているNAME_#フィールド項目の形式に自動的に変換されます。したがって、EBCDIC、

ZONED、PACKEDなどのフィールド形式を考慮することなく、ASCII値を引用符(')で囲んで入力します。

- 複数の「NAME_#: VALUE_#」条件が使用される場合は、各条件をコロン(:)で区切ります。

- REDEFINE_NAME

- 条件が成立した場合に実行する1つまたは複数のREDEFINEフィールド名を指定すると、条件文の作成が完了します。
- 条件文に値を入力するときに値のタイプを指定することで、フィールド・タイプを使用せずに必要なタイプに変換することができます。
- ユーザー指定のフィールド・タイプは以下のとおりです。

タイプ	説明
!	NOT EQUALITYをチェックします。
'(empty)'	フィールドのデフォルト・タイプに変換します。
'A'	入力値をASCII文字のまま使用します。
'E'	EBCDIC文字に変換します。
'P' または 'UP'	パック10進数または符号なしパック10進数形式に変換します。
'Z' または 'UZ'	ゾーン10進数または符号なしゾーン10進数形式に変換します。
'G'	GRAPHIC(2バイト)文字に変換します。
'H'	入力値を16進数に変換します。(例: H"123" → 0x7B)
'X'	入力した2バイト値を1バイトの16進数に変換します。(例: X"12AB" → 0x12 0xAB)
'T' (Zoned、Packed、Nationalのみサポート)	フィールド・タイプを確認します。(例: T"ZONED"/T"PACKED"/T"NATIONAL" → フィールドがゾーン/パック10進数または国別文字であるかを確認)

- パック10進数またはゾーン10進数の条件値を指定する場合は、そのフィールドが符号付き10進数と符号なし10進数のどちらの形式を使用するかを考慮して条件値のタイプを指定する必要があります。

たとえば、パック10進数フィールドの最終桁のHex値が'0x0C'(正数)または'0x0D'(負数)に指定されている場合は、'P'タイプを指定して符号付きパック10進数形式に条件値を変換します。最後のHex値が'0x0F'である場合は、'UP'タイプを指定して符号なしパック10進数形式に変換します。

ゾーン10進数の条件値を指定する場合は、ゾーン10進数フィールドの最終桁のHex値が'0xC0'(正数)または'0xD0'(負数)に指定されている場合、'Z'タイプを指定して符号付きゾーン10進数形式に条件値を変換する必要があります。最後のHex値が'0xF0'である場合は、'UZ'タイプを指定して符号なしゾーン10進数形式に変換します。

以下は、REDEFINES文が含まれているコピーブックに\$\$COND条件文を使用した例です。

```
01 A. 30.
    05 B      PIC X(01).
    05 C      PIC X(05).
    05 D      REDEFINES C.
    10 E      PIC X(02).
    10 F      PIC X(03).**
$$COND : B : "A" : D**
```

cobgenschツールは、\$\$COND文の情報を参照して、**dsmigin**ツールが使用できる移行スキーマを作成します。

以下は、作成された移行スキーマの例です。

```
* Schema Version 7.1
L1, 01, A, NULL, NULL, 0, 1:1,
L2, 05, B, EBC_ASC, NULL, 1, 1:1,
L3, 05, C, EBC_ASC, NULL, 5, 1:1,
L4, 05, D, NULL, NULL, 0, 1:1, # REDEFINES C
L5, 10, E, EBC_ASC, NULL, 2, 1:1,
L6, 10, F, EBC_ASC, NULL, 3, 1:1,

* Condition
**L2, "A", ( L1 L2 L4 L5 L6 )**
L0, "\0", ( L1 L2 L3 )
```

このように作成されたスキーマ・ファイルを使用してデータセットのレコードを移行すると、データセットの各レコードのフィールドBの位置にある値に従って適切な移行ルールが適用され、データセットが移行されます。

上記例では、コピーブックで作成した条件以外に、もう1つの条件が作成されています。最終行に作成された「L0」というラベルを持つ条件は、デフォルトで実行される条件文です。

3.5.4. 複数のレイアウトを使用するデータセット

複数のレイアウトを使用するデータセットは、REDEFINES文を処理する方法と同様に条件文を作成します。

```
01 AAA-A.
    03 BBB-A.
        05 DATA-1  PIC X(01).
        05 DATA-2  PIC X(04).
        05 DATA-A  REDEFINES DATA-2.
            07 DATA-A1  PIC 9(02).
            07 DATA-A2  PIC X(02).
01 AAA-B.
    03 BBB-B.
```

```

05 DATA-3    PIC    9(01).
05 DATA-4    PIC    9(04).
05 DATA-D    REDEFINES DATA-4.
    07 DATA-D1    PIC    9(01).
    07 DATA-D2    PIC    X(03).

```

上記のように01レベルの項目が複数存在する場合、2番目からのレイアウトを使用するには、REDEFINES文の処理と同様に\$\$COND文を使用する必要があります。

以下は、特定のレイアウトを使用するための\$\$COND文を作成する方法です。

```
$$COND : DATA-3 : "B" : AAA-B
```

以下は、2番目のレイアウトにあるREDEFINES文と一緒に指定するために作成した\$\$COND文です。

```
$$COND : DATA-3 : "D" : AAA-B DATA-D
```

以下は、**cobgensch**ツールを使用して作成したスキーマ・ファイルです。

```

* Schema Version 7.1
L1, 01, AAA-A, NULL, NULL, 0, 1:1,
L2, 03, BBB-A, NULL, NULL, 0, 1:1,
L3, 05, DATA-1, EBC_ASC, NULL, 1, 1:1,
L4, 05, DATA-2, EBC_ASC, NULL, 4, 1:1,
L5, 05, DATA-A, NULL, NULL, 0, 1:1, # REDEFINES DATA-2
L6, 07, DATA-A1, U_ZONED, NULL, 2, 1:1,
L7, 07, DATA-A2, EBC_ASC, NULL, 2, 1:1,

L8, 01, AAA-B, NULL, NULL, 0, 1:1,
L9, 03, BBB-B, NULL, NULL, 0, 1:1,
L10, 05, DATA-3, U_ZONED, NULL, 1, 1:1,
L11, 05, DATA-4, U_ZONED, NULL, 4, 1:1,
L12, 05, DATA-D, NULL, NULL, 0, 1:1, # REDEFINES DATA-4
L13, 07, DATA-D1, U_ZONED, NULL, 1, 1:1,
L14, 07, DATA-D2, EBC_ASC, NULL, 3, 1:1,

* Condition
L10, "B", ( L8 L9 L10 L11 )
L10, "D", ( L8 L9 L10 L12 L13 L14 )
L0, "\0", ( L1 L2 L3 L4 )

```

作成されたスキーマ・ファイルでは、01レベルを含む2つのレイアウトが含まれており、デフォルト条件を除く、\$\$COND文で作成した条件がすべて2番目の01レベル項目であるフィールドAAA-Bから始まることを確認できます。

このように生成されたスキーマ・ファイルを利用すると、REDEFINES文を使用したのと同様にフィールドDATA-3の位置にある値に応じて移行規則が適用され、複数のレイアウトを使用するデータセットを移行することができます。

第4章 アプリケーションの移行

本章では、CICSアプリケーションとバッチ・アプリケーションを移行する方法について説明します。

4.1. CICSアプリケーションの移行

CICSシステムのアプリケーション・プログラムをOpenFrame/OSCシステムで動作させるために移行します。アプリケーション・プログラムの移行は、EBCDIC文字で作成されたプログラム・ソースをASCII文字に変換するプロセスから始まります。以降のプロセスは、一般的なOSCアプリケーション・プログラムのコンパイルやデプロイ・プロセスと同じです。

CICSアプリケーションの移行プロセスは以下のとおりです。

1. EBCDICをASCIIに変換

EBCDIC文字をオープン環境で利用できるASCII文字に変換します。

2. 前処理

CICSアプリケーションをOpenFrame上で動作できるように、OpenFrameのツールを使用してCICSアプリケーションを修正します。

3. コンパイル

コンパイラを使って実行可能なバイナリを作成します。

4. デプロイ

生成されたバイナリをOpenFrameシステムにインポートします。

本節では、既存のCICSアプリケーションの特性によって、コンパイル時に行われる前処理やコンパイル時の注意事項、およびアプリケーションのデプロイ・プロセスについて説明します。

参考

EBCDIC文字をASCIIに変換するプロセスと変換時に発生する可能性のある問題点については、[「2.1. EBCDICからASCIIへの変換」](#)と[「付録 A. EBCDICからASCIIへの変換の注意点」](#)を参照してください。

4.1.1. 前処理

前処理は、CICSのために作成されたCOBOLアプリケーション・プログラムのソース・ファイルをOpenFrameが提供するツールを使用してOpenFrame COBOLコンパイラでコンパイルできるように変換することです。

CICSアプリケーションの前処理に使用されるツールは以下のとおりです。

1. ofcbpp

メインフレームCOBOLでのみサポートされる構文を前処理します。

OpenFrame環境でCOBOL CICSアプリケーションをコンパイルするために、ofcbppツールを使用して前処理を行います。ofcbppツールは、COBOLの文法をチェックし、参照されるコピーブックを展開するときに使用されます。また、特定の構文を置き換えるか挿入するときにも使用されます。たとえば、ユーザーのミスで終了期間が指定されていない場合、--add-miss-periodオプションを使用して構文エラーを回避し、終了を意味するSTOP RUNを、RETURNを意味するGOBACKに置き換えるために—conv-stop-runを使用します。

以下は、COBTEST.cobファイルでコピーブック・パスを指定し、--conv-stop-runを適用して前処理する例です。

```
$ ofcbpp -i COBTEST.cob -o ofcbpp_COBTEST.cob -copypath $PWD/COPYBOOK
--conv-stop-run
```

参考

ofcbppツールの使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。

2. osccobprep

osccobprepツールは、CICSでサポートされる例外的なCOBOL文法を前処理します。また、OSC COBOLアプリケーション・プログラムの文法を前処理します。たとえば、BLLセルは、UNIX環境のコンパイラを使用してコンパイルすると、メモリ・アクセス・エラーが発生するため、osccobprepを使用して前処理する必要があります。

以下は、OSCTEST.cobファイルでBLLセルを削除する前処理を実行する例です。

```
$ osccobprep -b OSCTEST.cob
```

3. dliprep

dliprepツールは、OSCシステムでHiDBにアクセスするために使用されるEXEC DLIコマンドが含まれているCOBOLソースを前処理します。

以下は、EXEC DLIコマンドが含まれているCOBOLソース・ファイルを前処理する例です。

```
$ dliprep DLITEST.cob
```

4. osccbldpp

osccbldppツールは、OSCシステムでリソースにアクセスするために使用されるEXEC CICSコマンドが含まれているCOBOLソースを前処理します。

以下は、EXEC CICSコマンドが含まれているCOBOLソース・ファイルを前処理する例です。

```
$ osccbldpp OSCTEST.cob
```

注

CICSアプリケーションを移行する場合は、osccblppツールを使用してすべてのソースを前処理する必要があります。ただし、ofcbpp、osccobpp、dliprepツールは必要に応じて使用してください。

4.1.2. コンパイル

前処理したアプリケーションのプログラム・ソースを基に、COBOLコンパイラを使って共有オブジェクトを作成します。既存のアプリケーション・プログラムのCOBOL文法の特徴に合わせてオプションを適用します。

参考

コンパイラを使用した共有オブジェクトの作成に関する詳細は、COBOLコンパイラの各ベンダーのマニュアルを参照してください。

4.1.3. デプロイ

OpenFrameの**osctdlupdate**ツールを使用して、コンパイルされたバイナリを以下の手順でデプロイおよび適用します。TDL(Tmax Dynamic Library)が設定済みの場合には、手順5に進んでください。

1. OSCアプリケーション・サーバーの環境設定ファイルの[GENERAL]セクションのTDLDIR項目を設定します。TDLDIR項目に設定したディレクトリの下に、modおよびrunディレクトリがない場合は、直接作成してください。TDLDIR項目を設定しない場合、OSCシステムがデフォルトで\${OPENFRAME_HOME}/osc/region/{リージョン名}/{TDLディレクトリ}を参照します。
2. TDLDIRディレクトリ配下のconfigディレクトリに、tdl.cfgファイルを作成します。OpenFrameシステムをインストール時にデプロイされたtdl.cfgサンプル・ファイルを使用します。
3. **osctdlinit**ツールを実行します。TDLDIRディレクトリに存在するtdl.cfgファイルの設定に従って初期化が行われます。
4. コンパイルされたアプリケーションをTDLDIRディレクトリ配下のmodディレクトリに保存します。
5. OSCリージョン名とプログラム名を指定して**osctdlupdate**ツールを実行し、アプリケーションをデプロイします。

以下は、OSCTESTというプログラムをアップデートする例です。(プログラム名は、拡張子「.so」が省略されています)

```
$ osctdlupdate OSC000001 OSCTEST
```

参考

1. TDLについての詳細は、『Tmax プログラミングガイド(ダイナミックライブラリ)』を参照してください。
 2. osctdlrmツールを使用して、使用中の共有メモリ情報を削除することができます。共有メモリ情報を削除した後再作成するときは、osctdlinitツールを実行します。各ツールについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。
-

4.2. バッチ・アプリケーションの移行

バッチ・アプリケーションの移行プロセスには、メインフレームで使用されていたバッチ・アプリケーションをOpenFrameで実行できるようにアプリケーション・ソースを変換する前処理とコンパイルが含まれます。

以下は、バッチ・アプリケーションの移行プロセスです。

1. EBCDICをASCIIに変換

EBCDIC文字をオープン環境で使えるASCII文字に変換します。

2. 前処理

バッチ・アプリケーションをOpenFrame上で動作できるように、OpenFrameのツールを使用してバッチ・アプリケーションを修正します。

3. コンパイル

コンパイラを使って実行可能なバイナリを作成します。

4. デプロイ

生成されたバイナリをOpenFrameシステムにインポートします。

Oracleをデータベースとして使用する場合は、一般的な移行プロセスに加えて、DB2 SQL文をOracle SQL文に変換する作業が必要です。本節では、COBOLとPL/Iの言語ごとに、前処理、コンパイル、デプロイの各プロセスとコンパイル時の注意事項について説明します。

参考

- 1.EBCDIC文字をASCIIに変換するプロセスと変換時に発生する可能性のある問題点については、[「2.1. EBCDICからASCIIへの変換」](#)と[「付録 A. EBCDICからASCIIへの変換の注意点」](#)を参照してください。
 - 2.DB2 SQL文をOracle SQL文に変換するプロセスについては、[「付録 B. DB2からOracleへの移行の注意点」](#)を参照してください。
-

4.2.1. COBOLバッチ・アプリケーション

本節では、COBOLバッチ・アプリケーションを移行する際の前処理、コンパイル、デプロイ・プロセスについて説明します。

前処理

OpenFrame環境でCOBOLバッチ・アプリケーションをコンパイルするために**ofcbpp**ツールを使用して前処理を行います。

ofcbppツールを使用してCOBOL文法をチェックし、参照されるコピーブックを展開しますが、**ofcob**のコンパイル時に前処理が実行されるため、必要に応じて**ofcbpp**を使用します。OpenFrame COBOLは、基本的にOpenFrame外部ファイル・ハンドラーを使用しますが、富士通 NetCOBOLを使用する場合は、OpenFrame外部ファイル・ハンドラーを使用するために**ofcbppf**前処理ツールを使用する必要があります。

以下は、TSTCOB.cobに参照されるコピーブック・パスを指定し、**ofcbpp**ツールを使用して前処理する例です。

```
$ ofcbpp -i TSTCOB.cob -o ofcbpp_TSTCOB.cob -copypath $PWD/COPYBOOK
```

以下は、TSTCOB.cobをサブモジュールとデバッグ・モードで**ofcbppf**ツールを使用して前処理する例です。

```
$ ofcbppf -s -t -i TSTCOB.cob -o ofcbppf_TESTCOB.cob
```

参考

ofcbppツールと**ofcbppf**ツールについての詳細は、『OpenFrame COBOLユーザーガイド』を参照してください。

コンパイル

COBOLバッチ・アプリケーションは、コンパイラー・ベンダーによって前処理プロセスが異なる場合があります。本節では、各コンパイラーのバッチ・アプリケーションのコンパイル方法について説明します。

● OpenFrame COBOLコンパイラー

外部ファイル・ハンドラーをサポートするOpenFrame COBOLコンパイラーは、ファイル入出力を前処理することなく、外部ファイル・ハンドラーを使用してOpenFrameのtextfh(Tmax外部ファイル・ハンドラー)モジュールをリンクし、呼び出す関数を指定することにより、データセットの入出力を使用することができます。

外部ファイル・ハンドラーを指定するために、OpenFrame COBOLで別途指定するオプションはありません。コンパイル時に libtextfh.so をリンクする必要があり、OpenFrame COBOLで指定した関数名で呼び出すためには、\$OPENFRAME_HOME/lib/libcobsw.so が libswofcob.so にリンクする必要があります。

以下は、OpenFrame COBOLコンパイラーを使用する例です。

```
ofcob -U -o COBSAMPLE.so COBSAMPLE.cob -L$OPENFRAME_HOME/lib -ltextfh
```

● MF-COBOLコンパイラー

外部ファイル・ハンドラーをサポートするMF-COBOLコンパイラーは、ファイル入出力を前処理することなく、外部ファイル・ハンドラーを使用してOpenFrameのtextfh(Tmax外部ファイル・ハンドラー)モジュールをリンクし、呼び出す関数を指定することにより、データセットの入出力を使用することができます。

外部ファイル・ハンドラーを指定するためには、-C'CALLFH"TEXTFH"オプションと-C'ASSIGN"EXTERNAL"オプションを追加して外部ファイル・ハンドラーが使用する関数を指定し、libtextfh.soをリンクする必要があります。OpenFrame COBOLで指定した関数名で呼び出すためには、\$OPENFRAME_HOME/lib/libcobsw.soがlibswmfcob.soにリンクする必要があります。

textfhモジュールではなく、tcobfhモジュールを使用できますが、MF-COBOLを使用する場合は、以下の例のように外部ファイル・ハンドラーを使用することをお勧めします。

以下は、MF-COBOLコンパイラーを使用する例です。

```
cob32 -zvP -U -t -o COBSAMPLE.so COBSAMPLE.cob  
-C'CALLFH"TEXTFH"' -C'ASSIGN"EXTERNAL"' -L$OPENFRAME_HOME/lib -ltextfh
```

● NET-COBOLコンパイラー

外部ファイル・ハンドラーをサポートしない.NET COBOLコンパイラーなどのCOBOLコンパイラーでは、**ofcbppf**ツールを使用してファイル入出力を前処理する必要があり、コンパイル時にOpenFrameのtcobfh(Tmax COBOLファイル・ハンドラー)モジュールをリンクする必要があります。

.NET COBOLコンパイラーを使用する場合は、以下の例のように**ofcbppf**ツールを使用して前処理してから、ライブラリのlibaimap.so、libaimacpin.so、libndbdml.soをリンクします。

以下は、.NET COBOLコンパイラーを使用する例です。

```
cobol -dy -shared  
-WC",STD1(JIS1),NOALPHAL,SRF(FIX),NSPCOMP(ASP),DLOAD,RSV(V125),CHECK,MODE(CCVS)"  
-o COBSAMPLE.so COBSAMPLE.cob -L$OPENFRAME_HOME/lib -laimap -laimacpin -lndbdml
```

参考

コマンドの詳細については、各コンパイラー・ベンダーが提供するマニュアルを参照してください。

デプロイ

新しいバージョンのプログラムまたはモジュールをOpenFrameシステムに動的に適用するときは、**dlupdate**ツールを使用します。**dlupdate**ツールは、モジュールによって実行中のプログラムとアプ

リケーションに影響を与えないために、既存のモジュールをバックアップして、実行されるプログラムとアプリケーションが新しいモジュールを使用できるようにします。

以下は、ツールの使用方法と項目についての説明です。

```
dlupdate source_path dataset_name
```

項目	説明
source_path	新しく適用されるモジュールのパスです。
dataset_name	既存のモジュール・パスに存在するPDSです。

dlupdateによってバックアップされたモジュールは、**dlclean**ツールを使用して削除することができます。ユーザーが**dlclean**ツールを直接実行しなくても、実行中のジョブがない場合は、**dlclean**ツールが定期的に行われ、バックアップされたモジュールを削除します。

参考

dlupdateツールとdlcleanツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

4.2.2. PL/I バッチ・アプリケーション

本節では、PL/Iバッチ・アプリケーションを移行する際のコンパイル、デプロイのプロセスについて説明します。

コンパイル

外部ファイル・ハンドラーをサポートするOpenFrame PL/Iは、ファイル入出力を前処理することなく、コンパイル時に外部ファイル・ハンドラーを使用してOpenFrameのtextfh(Tmax外部ファイル・ハンドラー)モジュールのライブラリ libtextfh.soをリンクし、呼び出す関数を指定することにより、データセットの入出力を使用することができます。

以下は、OpenFrame PL/Iコンパイラを使用する例です。

```
$ ofpli -o PLITEST.so PLITEST.pli -L$OPENFRAME_HOME/lib -ltextfh
```

参考

ofpliツールの詳細については、『OpenFrame PL/Iユーザーガイド』を参照してください。

OpenFrame PL/Iの前処理ツールは別途提供されませんが、中間ファイルを作成できるオプションがあります。--save-tempsを使用すると、作成するコードの特性に応じて .mpまたは .ppファイルが作成されます。PL/Iでは、メインプログラムとサブプログラムに分けられており、JCLで呼び出されるPL/Iアプリケーションは、OPTIONS(MAIN)が指定されている必要があります。

OpenFrame PL/Iでは、システムに応じてオプションを指定することができます。OSCで実行されるアプリケーションは --system-cicsを、OSIで実行されるアプリケーションは --system-imsを指定します。

デプロイ

新しいバージョンのプログラムまたはモジュールをOpenFrameシステムに動的に適用するときは、**dlupdate**ツールを使用します。**dlupdate**ツールは、モジュールによって実行中のプログラムおよびアプリケーションに影響を与えないために、既存のモジュールをバックアップして、実行されるプログラムおよびアプリケーションが新しいモジュールを使用できるようにします。

以下は、ツールの使用方法と項目についての説明です。

```
dlupdate source_path dataset_name
```

項目	説明
source_path	新しく適用されるモジュールのパスです。
dataset_name	既存のモジュール・パスに存在するPDSです。

dlupdateによってバックアップされたモジュールは、**dlclean**ツールを使用して削除することができます。ユーザーが**dlclean**ツールを直接実行しなくても、実行中のジョブがない場合は、**dlclean**ツールが定期的に実行され、バックアップされたモジュールを削除します。

参考

dlupdateツールと**dlclean**ツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

付録 A. EBCDICからASCIIへの変換の注意点

OpenFrameを使用してメインフレーム・システムからオープン・システム環境にリソースを移行するには、リソースをEBCDICからASCIIに変換する必要があります。

EBCDICデータをASCIIデータに変換するときにさまざまな問題が発生しますが、特に、COBOLで作成したアプリケーション・プログラムのソース・ファイルを変換する際に発生する場合があります。以下は、ソース・ファイルの変換時に発生する問題のうち、深刻度の高い3つの問題について説明します。

- 16進数値の処理
- 文字ソート順の処理
- 2バイト・スペースの処理

本節では、例を挙げて問題の原因や対処方法について説明します。

A.1. 16進数値の処理

以下は、16進数値を使用して作成されたロジックが含まれたCOBOLソース・プログラムをdsmininツールを使用してASCIIに変換する例です。

```
01  WORK06-AREA.
    05  W06-KOUZA-NO.
        10  FILLER                PIC X(01)  VALUE  X'F1'.
        10  W06-NO-1              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F2'.
        10  W06-NO-2              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F3'.
        10  W06-NO-3              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F4'.
        10  W06-NO-4              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F5'.
        10  W06-NO-5              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F6'.
        10  W06-NO-6              PIC X(01).
        10  FILLER                PIC X(01)  VALUE  X'F7'.
        10  W06-NO-7              PIC X(01).
```

上記のファイルは、COBOLで作成されたソース・ファイルを、一般的な変換方法を使用してEBCDIC文字をASCII文字に変換した状態を示しています。上記のファイルは、変換が正常に完了されたかのように見えますが、以下のような問題が含まれている可能性があります。

以下は、上記例の一部を示しています。

```
10 FILLER                                PIC X(01)  VALUE  X'F1'.
```

上記の行では、EBCDIC文字がASCII文字に正常に変換されていますが、EBCDICで作成されたCOBOLプログラムに実装された業務ロジックは正常に変換されていない可能性があります。

たとえば、ソース・ファイルでX'F1'がF1という文字を表すのではなく、X'F1'が指すEBCDIC値の1であると想定します。一般的な文字セットの変換プロセスではEBCDIC値の1を処理できないため、その値を手動で変更する必要があります。ソース・ファイルをASCIIに変換した後、以下のようにASCII値の1に該当する値の31にソースを修正します。

```
10  FILLER                PIC X(01)  VALUE  X'31'.
```

以下は、**dsmigin**ツールを使用して上記の最初のサンプル・ファイルと同じソース・ファイルをASCIIに変換した後、文字を数値に変換した例です。

```

01  WORK06-AREA.
      05  W06-KOUZA-NO.
            10  FILLER                                PIC X(01)  VALUE  X  '31'.
            10  W06-NO-1                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '32'.
            10  W06-NO-2                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '33'.
            10  W06-NO-3                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '34'.
            10  W06-NO-4                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '35'.
            10  W06-NO-5                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '36'.
            10  W06-NO-6                              PIC X(01).
            10  FILLER                                PIC X(01)  VALUE  X  '37'.
            10  W06-NO-7                              PIC X(01).

```

16進数値の処理時に発生する問題は、プログラムに16進数値が直接記述されており、文字を変換する際に問題が発生するのが一般的ですが、その逆の場合もあります。

01	YEAR-TABLE.									
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{JJJJJJ'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{JJJJJJ'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{{{JJJJ'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{{{{{JJJ'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{{{{{{{JJ'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{{{{{{{{{J'.			
	05	FILLER		PIC	X(12)	VALUE	'{{{{{{{{{{{{{{{{{{'.			
	05	FILLER		PIC	X(12)	VALUE	'A{{{{{{{{{{{{{{{{'.			
	05	FILLER		PIC	X(12)	VALUE	'AA{{{{{{{{{{{{{{{{'.			

05	FILLER	PIC	X(12)	VALUE	'AAA{{{{{{{{{{'.
05	FILLER	PIC	X(12)	VALUE	'AAAA{{{{{{{{{{'.
05	FILLER	PIC	X(12)	VALUE	'AAAAA{{{{{{{{{{'.

上記の例で「{」という文字は、プログラム・ロジックで文字として使用されたのではなく、「{」文字の16進数値に該当するX'C0'文字が指すZD(ゾーン10進数)値を意味します。

この場合は、メインフレームのZD値X'C0'に該当するASCIIのZD値はX'30'であるため、X'30'に該当するASCII文字の0に変更すると、ユーザー・アプリケーションを正常に変換できます。ただし、EBCDICソース・データをASCIIデータに変換した後にソースを修正することをお勧めします。

01	YEAR-TABLE.				
05	FILLER	PIC	X(12)	VALUE	'000000qqqqqq'.
05	FILLER	PIC	X(12)	VALUE	'0000000qqqqq'.
05	FILLER	PIC	X(12)	VALUE	'00000000qqqq'.
05	FILLER	PIC	X(12)	VALUE	'000000000qqq'.
05	FILLER	PIC	X(12)	VALUE	'0000000000qq'.
05	FILLER	PIC	X(12)	VALUE	'00000000000q'.
05	FILLER	PIC	X(12)	VALUE	'000000000000'.
05	FILLER	PIC	X(12)	VALUE	'100000000000'.
05	FILLER	PIC	X(12)	VALUE	'110000000000'.
05	FILLER	PIC	X(12)	VALUE	'111000000000'.
05	FILLER	PIC	X(12)	VALUE	'111100000000'.
05	FILLER	PIC	X(12)	VALUE	'111110000000'.

前述した16進数値の処理問題は、ユーザーが作成したプログラム・ソースの16進数値を意味するのか、その値が示すEBCDIC文字(または、逆の場合も該当する)を意味するのかが、文字セットの変換プロセスでは区別できません。

ソース・ファイルの16進数値を把握するには、COBOLで作成されたプログラム・ソースはCOBOL文法を、BMSマップの場合はBMSマクロ文法を適用して分析する必要があります。

参考

現在は、この問題を解決するためにTmaxSoftのコンサルタントが移行に直接参加してソース・ファイルを分析しています。今後、プログラム単位で自動的に分析できるツールを開発する予定です。

A.2. 文字ソート順の処理

EBCDIC文字をASCII文字に変換したとき、文字変換に問題がないように見える場合でも、各文字セットの固有の特性により、実際にユーザー・プログラムを変換して使用する際に業務ロジックに問題が発生することがあります。

本節では、文字セットの固有の特性によって発生する可能性のあるソート順問題について説明し、その対処方法について説明します。

以下は、メインフレームで使用しているCOBOLソースをASCII文字に変換したソースの一部です。

```
IF      W01-XX      <=      '99'      THEN
      MOVE      'Y'      TO      W01-CC
ELSE
      MOVE      'N'      TO      W01-CC
END-IF.
```

参考

EBCDICからASCIIへのソース変換は正常に行われましたが、ユーザー・プログラムに関してはビジネスロジックでエラーが発生する可能性があります。

以下は、EBCDIC文字とASCII文字のソート順です。

- EBCDIC: $a < z < A < Z < 0 < 9$
- ASCII: $0 < 9 < A < Z < a < z$

上記の例では、W01-XXの値をAAと想定しています。プログラムがメインフレーム上で動作する場合はW01-CCにYが設定され、UNIX上で動作する場合はW01-CCにNが設定されます。

そのため、メインフレームの業務ロジックのまま変換するには、以下のようにソースを修正する必要があります。

```
IF      W01-XX      <      'zz'      THEN
      MOVE      'Y'      TO      W01-CC
ELSE
      MOVE      'N'      TO      W01-CC
END-IF.
```

業務システム内の文字ソート順の処理時に発生する問題は簡単ではありませんが、ユーザー・プログラムを修正することで解決することができます。ただし、業務システム内の問題ではなく、一般のユーザーにも影響を与える可能性のある問題は、深刻な混乱を引き起こす可能性があります。

実際に業務システムを開発および運用するシステム・エンジニアは、EBCDICでは $ZZ < 99$ 、ASCIIでは $99 < ZZ$ であることを把握しており、文字セットの変換時にソート順を変更する必要があることを認識していますが、一般のユーザーがこれらの情報を常に把握しているのは容易ではありません。

以下は、業務画面を変換する例です。

```
[User Address List]
```

```
-----
```

```
      ID : AAAAAAAAAA
```

```
      ID              NAME              ADDRESS
```

AAAAAAAA	SUZUKI	TOKYO
BBBBBBBB	WATANABE	OSAKA
CCCCCCCC	MORI	FUKUOKA
HHHHHHHH	YAMADA	NAGOYA
LLLLLLLL	TANAKA	SAPPORO
MMMMMMMM	SHIMA	KYOTO
NNNNNNNN	KOBAYASHI	SENDAI
XXXXXXXX	ONO	KANAZAWA
ZZZZZZZZ	MIZUNO	CHIBA

<F1> Menu <F2> Prev <F3> Next <Enter> Search

上記の業務ロジックは、メインフレーム環境では、IDの最小値であるAAAAAAAAを使用してすべてのID一覧を検索することができますが、オープン環境では、AAAAAAAAを入力しても、すべてのID一覧を検索することができません。

また、11111111というIDがあると想定した場合、メインフレーム環境では<F3>キーを押して次の画面を検索すると、11111111の結果が表示されますが、オープン環境では、ZZZZZZZZ以降のID一覧は表示されません。つまり、オープン環境についての知識がなく、既存のメインフレーム・ユーザーは、11111111というID情報が実際にプログラムに存在するとしても、それを認識することは難しいです。

以下は、オープン環境ですべてのID一覧を検索(AAAAAAAAAの代わりに00000000を使用)する例です。

[User Address List]		
ID : 00000000		
ID	NAME	ADDRESS
11111111	OGAWA	KAWASAKI
88888888	SHIBATA	YOKOHAMA
AAAAAAAA	SUZUKI	TOKYO
BBBBBBBB	WATANABE	OSAKA
CCCCCCCC	MORI	FUKUOKA
HHHHHHHH	YAMADA	NAGOYA
LLLLLLLL	TANAKA	SAPPORO
MMMMMMMM	SHIMA	KYOTO
NNNNNNNN	KOBAYASHI	SENDAI

<F1> Menu <F2> Prev <F3> Next <Enter> Search

文字ソート順の問題も16進数値の処理問題と同様に、プログラムを直接確認してから判断しなければならぬため、現在は専門のコンサルタントが問題の解決にあたっています。

A.3. 2バイト・スペースの処理

メインフレーム環境では、2バイト・スペースはX'4040'、つまり1バイト・スペースのX'40'を2つ合わせた値と同一値として認識されます。

```
10  W-K-00.  
    20  W-K-00-1          PIC  X(01).  
    20  W-K-00-2          PIC  X(01).  
10  W-K-01      REDEFINES  W-K-00.  
    20  W-K-01-1          PIC  G(01).  
  
* . . .  
  
MOVE SPACE TO W-K-01-1.  
  
IF W-K-00-1 = SPACE THEN  
    DISPLAY 'DOUBLE BYTE SPACE = SINGLE BYTE SPACE * 2'  
END-IF.
```

上記のCOBOLプログラムがメインフレーム環境で実行される場合は、「DOUBLE BYTE SPACE = SINGLE BYTE SPACE *2」というメッセージが出力されますが、オープン環境で実行される場合は、そのメッセージが出力されません。

そのため、メインフレームで2バイト・スペースを処理する方法は、オープン環境に移行されたときにユーザー・プログラムのロジックが変更される問題をもたらします。

このような問題が発生する理由は、韓国語の場合、オープン環境にデータを移行する際、EUC-KRで2バイト文字を変換しますが、EUC-KR文字セットでは、2バイト・スペースはX'8140'、1バイト・スペースはX'20'という値にマッピングされるため、「シングル・バイト・スペース+シングル・バイト・スペース=ダブル・バイト・スペース」という式が成立しないためです。

このような問題は、コンパイラーが提供する機能を使用して簡単に解決できる場合もありますが、そうでない場合もあります。

OpenFrameでは、オープン環境で2バイト・スペースを使用しないことを解決策として提示しています。なるべく、2バイト・スペースを使用しないようにし、2バイト・スペースを使用する必要がある場合は、2つの1バイト・スペースで2バイト・スペースを代替する方法を使用してください。COBOLコンパイラーは、2バイト・スペースを無視するオプションを提供しているため、このような問題は簡単に解決することができます。

OpenFrameのデータをメインフレーム環境に送信する場合は、2バイト・スペースを使用する必要があります。そのため、2つの1バイト・スペースを使用して代替する方法は使用できません。このような場合は、OpenFrameのCPMで処理できるようにサポートしています。

付録 B. DB2からOracleへの移行の注意点

メインフレームの業務システムをオープン環境に移行する際、DB2データベースをOracleデータベースに変更する場合は、業務システムのソース変更が必要になることがあります。

本付録では、DB2データベースからOracleデータベースへの移行時に発生するアプリケーションの移行問題について説明します。

参考

詳細については、各ベンダーの移行ガイドを参照してください。

以下は、データベースの移行に必要なアプリケーションの変更事項です。

- **OPTIMIZE ... FOR FETCH**

- タイプ

```
OPTIMIZE FOR 1 ROWS FOR FETCH ONLY
```

- 変更 : コメントアウト

- **SET CURRENT PACKAGE**

- タイプ

```
EXEC SQL
    SET CURRENT PACKAGE = "文字"
END-EXEC.
```

- 変更 : コメントアウト

- **SET :[Host Variable] = CURRENT TIMESTAMP**

- タイプ

```
EXEC SQL
    SET :[Host Variable] = CURRENT TIMESTAMP
END-EXEC.
```

- 変更 : コメントアウトとクエリの挿入

```
EXEC SQL
    SELECT TO_CHAR(SYSTIMESTAMP, 'YYYY.MM.DD.HH24.MI.SS.FF6')
    INTO :[Host Variable]
    FROM DUAL
END-EXEC.
```

● CURRENT TIMESTAMP

– タイプ

```
[DB Column] = CURRENT TIMESTAMP
```

– 変更

```
[DB Column] = SYSTIMESTAMP
```

● EVALUATE

– タイプ

```
EVALUATE  
    WHEN [条件]...  
END-EVALUATE.
```

– 変更

```
EVALUATE TRUE  
    WHEN [条件]...  
END-EVALUATE.
```

参考

EVALUATE句の条件がない場合は、MF-COBOLではエラーが返されるため、条件がないときはTRUEに代替する必要があります。

● SQLSTATE変数の定義

– タイプ

```
EXEC SQL  
    BEGIN DECLARE SECTION  
END-EXEC.
```

– 変更

- 上記の文がある場合は、文の中にSQLSTATE変数を挿入します。
- 上記の文がない場合は、SQLSTATE変数を定義します。

```
EXEC SQL  
    BEGIN DECLARE SECTION  
END-EXEC.  
  
01 SQLSTATE PIC X(5).  
  
EXEC SQL
```

```
END DECLARE SECTION
END-EXEC
```

- **SQLCODEの変換**

- Case 1

- タイプ

```
IF SQLCODE = -811
```

- 変更

```
IF SQLCODE = -2112
```

- Case 2

- タイプ

```
IF SQLCODE = -803
```

- 変更

```
IF SQLCODE = -1
```

- Case 3

- タイプ

```
WHEN -811 PERFORM [Section Name]
```

- 変更

```
WHEN -2112 PERFORM [Section Name]
```

- Case 4

- タイプ

```
WHEN -803 PERFORM [Section Name]
```

- 変更

```
WHEN -1 PERFORM [Section Name]
```

- [参考]**

```
ORA 02112, 00000, "PCC: SELECT..INTO returns too many rows"
ORA 00001, 00000, "unique constraint (%s.%s) violated"
```

- **SET :[Host Variable] = CURRENT DATE**

- タイプ

```
EXEC SQL
    SET :[Host Variable] = CURRENT DATE
END-EXEC.
```

- 変更

```
EXEC SQL
    SELECT TO_CHAR(SYSDATE, 'YYYY-MM-DD')
    INTO :[Host Variable] FROM DUAL
END-EXEC.
```

- **CURRENT DATE**

- タイプ

```
[DB Column] = CURRENT DATE
```

- 変更

```
[DB Column] = TO_CHAR(SYSDATE, 'YYYY-MM-DD')
```

- **SET :[Host Variable] = CURRENT TIME**

- タイプ

```
EXEC SQL
    SET :[Host Variable] = CURRENT TIME
END-EXEC.
```

- 変更

```
EXEC SQL
    SELECT TO_CHAR(SYSDATE, 'HH24.MI.SS')
    INTO :[Host Variable] FROM DUAL
END-EXEC.
```

- **VALUE関数**

- Case 1

- タイプ

```
SELECT VALUE(MAX([DB Column]), '')/VALUE(MIN([DB Column]), '')
```

- 変更

```
SELECT NVL(MAX([DB Column]), '')/NVL(MIN([DB Column]), '')
```

– Case 2

- タイプ

```
SELECT VALUE(SUM([DB Column]),0)
```

- 変更

```
SELECT NVL(SUM([DB Column]),0)
```

● DECIMAL関数

– Case 1

- タイプ

```
SELECT VALUE(SUM([DB Column]),DECIMAL(0,15,0))
```

- 変更

```
SELECT NVL(SUM([DB Column]),0)
```

– Case 2

- タイプ

```
SELECT VALUE(DECIMAL(0,15,0),SUM([DB Column]))
```

- 変更

```
SELECT NVL(0,SUM([DB Column]))
```

– Case 3

- タイプ

```
SELECT VALUE(MAX([DB Column]),DECIMAL(' ',15,' '))
```

- 変更

```
SELECT NVL(MAX([DB Column]),' ')
```

● INCLUDE SQLCA

SQLCABC/SQLERRML/SQLERRDを使用するが、INCLUDE SQLCAが指定されていない場合は、WORKING-STORAGE SECTIONに次の構文を挿入する必要があります。

```
EXEC SQL INCLUDE SQLCA END-EXEC.
```

● SQLCODE DISPLAY

デバッグ・モードでコンパイルする際、コンパイル対象ソースのSQL文に次の構文を挿入します。

```
DISPLAY '[SOURCE NAME]:LINE:[LINE NO]:[I/U/D/S]:SQLCODE:[SQLCODE]'.
```

項目	説明
SOURCE NAME	デバッグ・モードを適用するソース名です。
LINE NO	DISPLAY文を挿入する行番号です。
I/U/D/S	INSERT/UPDATE/DELETE/SELECTを示します。
SQLCODE	実行結果のSQLコードです。

付録 C. スキーマ・ファイルの構造

メインフレーム・データセットをOpenFrameデータセットに変換する場合は、ソース・データセットのレイアウトを分析することが最も重要です。そのため、COBOLコピーブックまたはPL/インクルード・ファイルを参照して、データセット変換スキーマ・ファイルを作成する必要があります。

本付録では、作成されたスキーマ・ファイル(v7.1)の構造とデータセットの変換に必要な項目について説明します。

C.1. スキーマ・ファイルの構造(v7.1)

OpenFrameでは、**cobgensch**、**pligensch**、**sqlgensch**ツールを使用してスキーマ・ファイルを作成することができます。

スキーマには、**一般変換スキーマ**と**動的変換スキーマ**があります。

● 一般変換スキーマ

```
* Schema Version 7.1
Label, Level, Field Name, Conversion Type, Sign Type, Length, Occurs, ODO Field

* Condition
Label, "Value", ( Label List )
```

● 動的変換スキーマ

動的変換スキーマには、一般変換スキーマに加えて、Redefine Group IDとRedefine Element IDが追加されています。以下は、動的変換スキーマ・ファイルの構造です。

```
* Schema Version 7.1 - Dynamic conversion
Label, Level, Field Name, Conversion type, Sign type, Length, Occurs, ODO Field,
Redefine Group ID, Redefine Element ID

* Condition
Label, "Value", ( Label List )
```

参考

1. cobgensch、pligensch、sqlgenschツールについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。
 2. 最新のOpenFrame/Baseツールのうちsqlgenschを使用して作成されたスキーマ・ファイルのバージョンは7.1バージョンと同じではないため、そのスキーマ・ファイルは移行に使用することができません。
-

以下は、スキーマ・ファイルの各項目についての説明です。

項目	説明
Label	フィールドの一意的ラベル値です。
Level	フィールドのレベルです。
Field Name	フィールド名です。
Conversion Type	フィールドのデータ型です。 – EBC_ASC : 1バイト文字 – COPY : バイナリ – PACKED : パック10進数 – U_PACKED : 符号なしパック10進数 – ZONED : ゾーン10進数 – U_ZONED : 符号なしゾーン10進数 – GRAPHIC : 2バイト文字
Sign Type	数字フィールドに対してCOBOLのSIGN IS句で定義された符号タイプです。デフォルト値はTRAILINGです。 – TRAILING : 2ビットの符号がフィールドの後に表示されます。 – LEADING : 2ビットの符号がフィールドの前に表示されます。 – TRAILING_SEPARATE : 1バイトの符号がフィールドの後に分離して表示されます。 – LEADING_SEPARATE : 1バイトの符号がフィールドの前に分離して表示されます。
Length	フィールドの長さです。
Occurs	繰り返し回数です。
ODO Field	Occurs Depending Onが指しているフィールド名です。
Redefine Group ID	REDEFINES文を使用したグループIDです。
Redefine Element ID	Redefine Group ID内の各エレメントのIDです。

COND文

スキーマ・ファイルの条件句は、コピーブックのCOND文によって作成されます。COND文の構造は以下のとおりです。


```
$$COND : FIELD_01 : VALUE_01 ( , VALUE02 ... ) [: FIELD_02 : VALUE_03 ( , VALUE04 ) ...  
] : REDEFINE_NAME [...]
```

「FIELD_#:VALUE_#」条件を複数指定する場合は、コロン(:)で区切ることが可能であり、各フィールドでは、条件値(VALUE)を複数比較することができます。条件値と一致すると、REDEFINES文が使用されます。

参考

REDEFINES文の詳細については、[「3.5.3. REDEFINES文が使用されたデータセット」](#)を参照してください。

条件句

\$\$COND文に対応するスキーマ・ファイルの条件句の構造について説明します。

```
L#1, "VALUE_01" "VALUE02" ... [ , L#2, "VALUE_03" "VALUE04 ... ], ( Label List )
```

項目	説明
Label(L#)	REDEFINES文で定義されたフィールドのラベル番号です。
条件値(VALUE_#)	REDEFINES文で定義されたラベルのフィールド値と比較される値(VALUE)です。一致する場合は、ラベル・リストの順でレイアウトが構成されます。 1つのラベルのフィールド値を複数の条件値と比較できます。そのとき、条件値はORで結合されていると考えられます。複数のラベルの条件文をコンマ(,)で区切って条件値と比較することもできます。
Label List	ラベルのフィールド値と条件値が一致するときに構成されるラベルの順序リストです。

C.1.1. ODO(Occurs Depending On)項目

ODO句は、他のフィールドに保存されている値を参照して、OCCURS回数を決めるときに使用できるオプション項目です。下記のコピーブックは、ODOオプションを使用した例です。

ODO句で指定するOCCURS回数は、最小値/最大値(1つだけが指定されている場合は最大値)を表しており、実際の繰り返し回数ではありません。ODO句を追加しない場合は、OCCURS回数のデフォルト値として1が設定されます。

参考

現在OpenFrameでは、レイアウト内に定義されているフィールドに対してのみODOフィールドを指定できます。レイアウト外に定義されたフィールドに対しては指定できません。

以下は、ODO項目が含まれているCOBOLコピーブックと作成されたスキーマ・ファイルの例です。

<COBOLコピーブック>

```
01 ROOT.  
  03 AAA PIC X(1).  
  03 ODO OCCURS 10 DEPENDING ON AAA.  
    05 BBB PIC S9(2).  
  03 CCC PIC X(1) OCCURS 5.
```

<スキーマ・ファイル>

```
* Schema Version 7.1  
L1, 01, ROOT, NULL, NULL, 0, 1:1,  
L2, 03, AAA, EBC_ASC, NULL, 1, 1:1,  
L3, 03, ODO, NULL, NULL, 0, 1:10, AAA  
L4, 05, BBB, ZONED, TRAILING, 2, 1:1,  
L5, 03, CCC, EBC_ASC, NULL, 1, 1:5,  
  
* Condition  
L0, "\0", ( L1 L2 L3 L4 L5 )
```

上記の例では、L0で始まる条件文は、デフォルトのレイアウト順序リストを示しています。詳細については、[「C.1.2. REDEFINES文」](#)を参照してください。

C.1.2. REDEFINES文

REDEFINES文が含まれているレイアウトを持つデータセットは、分岐条件に関する情報が必要です。

参考

REDEFINES文の詳細については、[「3.5.3. REDEFINES文が使用されたデータセット」](#)を参照してください。

以下は、REDEFINES文が使用されたCOBOLコピーブックとスキーマ・ファイルの例です。

<COBOLコピーブック>

```
01 ROOT.  
  03 AAA PIC X(2).  
  03 BBB PIC X(3).  
  03 BBB-1 PIC S9(3) REDEFINES BBB.  
  03 CCC PIC X(3).  
  03 CCC-1 PIC 9(5) COMP-3 REDEFINES CCC.
```

```

$$$COND : AAA : "AB" : BBB-1 ( multiple redefines can be defined here!)
$$$COND : CCC : T"PACKED" : CCC-1

```

<スキーマ・ファイル>

```

* Schema Version 7.1
L1, 01, ROOT, NULL, NULL, 0, 1:1,
L2, 03, AAA, EBC_ASC, NULL, 2, 1:1,
L3, 03, BBB, EBC_ASC, NULL, 3, 1:1,
L4, 03, BBB-1, ZONED, TRAILING, 3, 1:1, # REDEFINES BBB
L5, 03, CCC, EBC_ASC, NULL, 3, 1:1,
L6, 03, CCC-1, U_PACKED, NULL, 3, 1:1, # REDEFINES CCC

* Condition
L2, "AB", ( L1 L2 L4 L5 )
L5, T"PACKED", ( L1 L2 L3 L6 )
L0, "\0", ( L1 L2 L3 L5 )

```

上記の例で、コピーブックとスキーマ・ファイルの最初の条件文を比較すると、AAAというフィールド名に対応するのがL2(Label2)であり、このフィールドに該当する条件値が"AB"の場合にREDEFINES文が実行されるため、L1、L2、L4、L5の順にフィールド・レイアウトが構成されます。2番目の条件文を比較すると、CCCというフィールド名に対応するのがL5(Label5)であり、このフィールドに該当する条件値がT"PACKED"の場合、つまり、L5に対応するオフセット値がパック10進数である場合にREDEFINES文が実行されるため、L1、L2、L3、L6の順にフィールド・レイアウトが構成されます。最後のL0は、デフォルトで構成されるレイアウト順序リストを表します。

C.1.3. 条件値(VALUE)のタイプ

コピーブックの\$\$\$COND文に対応されるスキーマ・ファイルの条件句の条件値は、複数のタイプで指定して必要に応じてフィールド・タイプを変換することができます。この場合、条件値を囲む二重引用符(" ")の前にユーザー指定のタイプを指定できます。

以下は、ユーザー指定のフィールド・タイプです。

タイプ	説明
!(感嘆符)	NOT EQUALITYをチェックします。
'(empty)'	フィールドのデフォルト・タイプに変換します。
'A'	入力値をASCII文字のまま使用します。
'E'	EBCDIC文字に変換します。
'P' または 'UP'	パック10進数または符号なしパック10進数フォーマットに変換します。
'Z' または 'UZ'	ゾーン10進数または符号なしゾーン10進数フォーマットに変換します。
'G'	GRAPHIC(2バイト)文字に変換します。

タイプ	説明
'H'	入力値を16進数値に変換します。(例: H"123" → 0x7B)
'X'	入力した2バイト値を1バイトの16進数値に変換します。(例: X"12AB" → 0x12 0xAB)
'T'	フィールド・タイプを確認します。サポートするタイプは以下のとおりです。 – ZONED (ゾーン10進数) – PACKED (パック10進数) – GRAPHIC (2バイト文字) – CHAR (1バイト文字) (例: T"ZONED"→ フィールドがゾーン10進数であるかどうかを確認)

参考

条件値(VALUE)タイプについては、[「3.5.3. REDEFINES文が使用されたデータセット」](#)を参照してください。

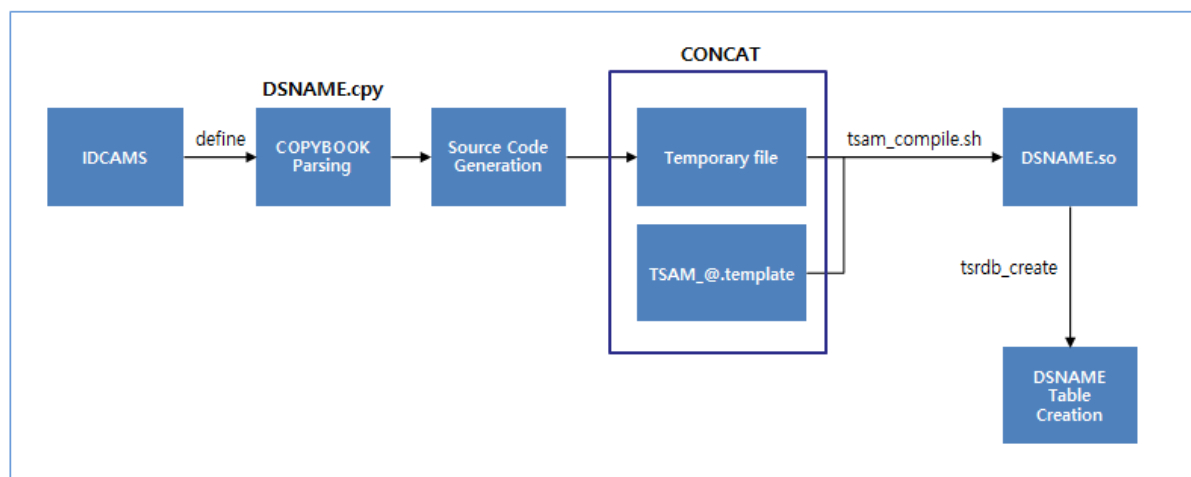
付録 D. TSAM RDB

本付録では、データをコピーブックのフィールドごとに変換して管理できるように、VSAMデータセットを定義する機能について説明します。

D.1. VSAMデータセットの定義プロセス

TSAM RDBでIDCAMSを使用してVSAMデータセットを定義するプロセスは以下のとおりです。

[図 D.1] VSAM データセットの定義プロセス



1. データセットと同じ名前のコピーブックを解析し、それにマッピングするソースコードを作成します。
2. 作成されたソースコードとテンプレート・ファイル(TSAM_@.template)を結合します。
3. 完成されたソース・ファイルをシェル・スクリプト(tsam_compile.sh)でコンパイルして、DSNAME.soを作成します。
4. DSNAME.soに定義されているtsrdb_create APIを呼び出して、データセットのテーブルを作成します。

D.2. TSAM RDBの特徴

本節では、TSAM RDBの主な特徴について説明します。

TSAM RDBはレコードをフィールド単位に分割した後、各フィールドのタイプに合わせてデータを変換してテーブルに保存します。

以下は、OCCURSフィールドのすべてのエレメントをデータベース表の1つの列として作成する例です。

<SAMPLE.KSDS.cpy>

```
01 REC.
    03 FLD-KEY PIC 9(10).
    03 FLD-OCC PIC X(10) OCCURS 5 TIMES.
    03 FLD-DAT PIC X(20).
    :
    :
```

```
$ idcams define CL -n SAMPLE.KSDS -o KS -k 10,0 -l 80,80 -0
```

COLUMN_NAME	TYPE	CONSTARINT
-----	-----	-----
FLD_KEY	NUMBER(10)	PRIMARY KEY
FLD_OCC	VARCHAR(50)	
FLD_DAT	VARCHAR(20)	
INDEX_NAME	TYPE	COLUMN_NAME
-----	-----	-----
SAMPLE_KSDS_KEY	NORMAL	FLD_KEY

以下は、OCCURSフィールドのエレメントごとに列を作成する例です。

<SAMPLE.KSDS.cpy>

```
01 REC.
    03 FLD-KEY PIC 9(10).
    03 FLD-OCC PIC X(10) OCCURS 5 TIMES.
    03 FLD-DAT PIC X(20).
    :
    :
```

```
$ idcams define CL -n SAMPLE.KSDS -o KS -k 10,0 -l 80,80
```

COLUMN_NAME	TYPE	CONSTARINT
-----	-----	-----
FLD_KEY	NUMBER(10)	PRIMARY KEY
FLD_OCC1	VARCHAR(10)	
FLD_OCC2	VARCHAR(10)	
FLD_OCC3	VARCHAR(10)	
FLD_OCC4	VARCHAR(10)	
FLD_OCC5	VARCHAR(10)	
FLD_DAT	VARCHAR(20)	

INDEX_NAME	TYPE	COLUMN_NAME
-----	-----	-----
SAMPLE_KSDS_KEY	NORMAL	FLD_KEY

基本的にコピーブックに定義されたフィールドに合わせて列が作成されますが、VB形式のレコードを処理するためにデータベース表の1つの列(BULK)でVSAMデータセットを作成することもできます。ただし、その場合は、一般フィールドとこのフィールドを再定義するフィールドだけで構成されたコピーブックが作成される必要があります。

以下は、1つの例(BULK)を持つVSAM KSDSデータセットを定義するために使用されるコピーブック・ファイルを作成し、そのコピーブックを使用してデータセットを定義する例です。

<BULK.KSDS.cpy>

```
01 REC.
    03 FLD-DAT PIC 9(80).
    03 FLD-BULK PIC X(80) REDEFINES FLD-DAT.
    :
    :
```

```
$ idcams define CL -n BULK.KSDS -o KS -k 10,0 -l 80,80
```

COLUMN_NAME	TYPE	CONSTRAINT
-----	-----	-----
FLD_DAT	VARCHAR(80)	
BULK_KSDS_COL	VARCHAR(10)	

INDEX_NAME	TYPE	COLUMN_NAME
-----	-----	-----
BULK_KSDS_KEY	FUNCTION-BASED NORMAL	- EXPRESSION COLUMN -

参考

再定義フィールドは、フィールド・タイプに関係なく、文字型の列が作成されます。

ISAMの場合は、既存のESQL TSAMと同様にVSAM KSDSが生成され、RDBバージョンの特徴に合わせてコピーブックが要求されます。ただ、一般的なVSAM KSDSの作成とは異なって、コピーブック・ファイル名の前に論理ボリューム名が付けられる必要があります。

以下は、TSAM RDBバージョンでISAMデータセットを定義するために使用されるコピーブック・ファイルとこのコピーブックを使用して**dscreate**ツールでISAMデータセットを定義する例です。

<DEFVOL_SAMPLE.ISAM.cpy>

```
01 REC.
    03 FLD-KEY PIC X(10).
    03 FLD-DAT PIC X(70).
```


:		
:		
\$ dscreate SAMPLE.ISAM -o IS -k 10 -p 0 -l 80 -v DEFVOL		
COLUMN_NAME	TYPE	CONSTARINT

FLD_KEY	VARCHAR(10)	PRIMARY KEY
FLD_DAT	VARCHAR(70)	
INDEX_NAME	TYPE	COLUMN_NAME

DEFVOL_SAMPLE_ISAM_KEY	NORMAL	FLD_KEY

D.3. 使用例

VSAMデータセットを定義する手順は以下のとおりです。

1. コピーブック・ディレクトリに、データセットと同じ名前のコピーブック・ファイル(例: SAMPLE.KSDS.cpy)を作成します。

<SAMPLE.KSDS.cpy>

```
01 REC.
  03 FLD-KEY  PIC 9(10).
  03 FLD-AKEY PIC X(10).
  03 FLD-DAT  PIC X(60).
  :
  :
```

2. **idcams**ツールまたはユーティリティでVSAMデータセットを定義します。

```
$ idcams define CL -n SAMPLE.KSDS -o KS -k 10,0 -l 80,80
```

3. データセット・テーブルと共有オブジェクト・ファイル(例: SAMPLE.KSDS.so)の作成を確認します。

COLUMN_NAME	TYPE	CONSTARINT

FLD_KEY	NUMBER(10)	PRIMARY KEY
FLD_AKEY	VARCHAR(10)	
FLD_DAT	VARCHAR(60)	
INDEX_NAME	TYPE	COLUMN_NAME

SAMPLE_KSDS_KEY	NORMAL	FLD_KEY

代替索引を定義する手順はデータセットを定義する手順と同じです。

以下は、VSAM KSDSデータセットの代替索引を定義する例とその結果の例です。

<SAMPLE.KSDS.cpy>

```
01 REC.
    03 FLD-KEY  PIC 9(10).
    03 FLD-AKEY PIC X(10).
    03 FLD-DAT  PIC X(60).
    :
    :
```

```
$ idcams define AIX -n SAMPLE.KSDS.AIX -r SAMPLE.KSDS -k 10,10 -l 80,80
```

COLUMN_NAME	TYPE	CONSTARINT

FLD_KEY	NUMBER(10)	PRIMARY KEY
FLD_AKEY	VARCHAR(10)	
FLD_DAT	VARCHAR(60)	
INDEX_NAME	TYPE	COLUMN_NAME

SAMPLE_KSDS_AIX_INDEX	NORMAL	FLD_AKEY
SAMPLE_KSDS_KEY	NORMAL	FLD_KEY

索引

C

CICSマップの移行, 8
cobgenschツール, 18, 26
COBOLバッチ・アプリケーション
 コンパイル, 37
 デプロイ, 38
 前処理, 37

D

dlcleanコマンド, 39, 40
dliprepコマンド, 34
dlupdateコマンド, 38, 40
dscreateツール, 20
dsmigin**ツ, 8
dsmiginツール, 5, 20, 24, 26
dsviewツール, 22

E

EBCDICからASCIIへの変換, 5
EBCDICデータ, 2
ESDS, 16

G

GDG, 15
gdgcreateツール, 21

I

idcamsツール, 23
IDCAMSユーティリティ, 24
ISAM, 15

J

JCLの移行, 11

K

KSDS, 16

L

LDS, 16
listcatツール, 22

M

mscadjustツール, 8
mscasmcツール, 8

O

ofcbppコマンド, 34, 37
OpenFrame移行, 1
osccblppコマンド, 34
osccobprepコマンド, 34
oscsdgenツール, 10
osctdlinitツール, 35
osctdlupdateツール, 35

P

PDS, 15
pdsgenツール, 20
PL/I バッチ・アプリケーション
 デプロイ, 40
PL/Iバッチ・アプリケーション
 コンパイル, 39
pligenschツール, 19

R

RDW, 21
REDEFINES文が使用されたデータセットの移行, 29
RRDS, 16

S

SAM, 15

V

VSAMデータセットの移行, 23
VSAMデータセットの移行方法, 15
 ESDS, 16

KSDS, 16

LDS, 16

RRDS, 16

あ

移行, 4

アプリケーションの移行, 3

データセットの移行, 3

データベースの移行, 4

リソースの移行, 2

アプリケーションの移行, 3

か

可変長データセットの移行, 28

固定長データセットの移行, 27

さ

制御ファイル, 11

静的変換, 25

た

動的変換, 25

データセットの移行, 3

データベースの移行, 4

は

非VSAMデータセットの移行方法, 15

GDG, 15

ISAMデータセット, 15

PDS, 15

SAMデータセット, 15

非VSAMデータセットの移行, 17

複数のレイアウトを使用するデータセットの移行, 31

ビッグバン(Big-Bang), 1

ら

リソース, 2

リソースの移行, 2

リホスト(Rehosting), 1